



الجمهورية العربية السورية

وزارة التعليم العالي والبحث العلمي

الجامعة الوطنية الخاصة

كلية الهندسة – قسم هندسة الحاسوب والاتصالات

مشروع تخرج أعد لنيل درجة الإجازة في هندسة الحاسوب والاتصالات بعنوان:

**CyberShield**

تطبيق مدعوم بالذكاء الاصطناعي لفحص الملفات والروابط

إعداد الطالبات:

هبة الفارس

أمني الأمين

آلاء حليبي

إشراف:

الدكتور محمود أيوب

الدكتور أحمد العبدو

العام الدراسي 2025 – 2026

## الإهداء

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

إلى من كان دعاؤهم نوراً يضيء الطريق، وصبرهم سنداً لا يميل، ومحبتهم وطناً نعود إليه كلما أثقلتنا الأيام...

إلى والدينا وأهلنا الأعزاء،

أنتم البداية الجميلة لكل إنجاز، وبكم كبرت الأحلام حتى أصبحت واقعة. نهديكُم ثمرة هذا الجهد المتواضع،

عرفاناً بما منحتمونا من حبٍ وعطاءٍ وثقةٍ لا تنتضب. نسأل الله أن يجزيكم عنا خير الجزاء، وأن يديمكم لنا

نعمةً لا تزول.

وإلى أصدقائنا الذين شاركونا تفاصيل الرحلة، وخففوا عنا عناء الطريق بابتسامةٍ صادقة، وكلمةٍ طيبة، ودعمٍ لم

ينقطع، لكم منا خالص الامتنان والمحبة.

وإلى الجامعة الوطنية الخاصة، وكلية الهندسة، وقسم هندسة الحاسوب والاتصالات، وإلى جميع أعضاء

الهيئة التدريسية والإدارية، الذين هيؤوا لنا بيئةً علميةً ثرية، وأسهموا في بناء معارفنا وصقل مهارتنا، فكان

لهم أثرٌ عظيم في مسيرتنا الأكاديمية.

وفي ختام هذا الإهداء، نتقدم بخالص التقدير والامتنان إلى الأستاذين المشرفين:

الدكتور أحمد العبدو – الدكتور محمود أيوب

لما قدماه لنا من علمٍ وتوجيهٍ وصبرٍ ومتابعةٍ مستمرة، فكانا خير مرشدين وخير داعمين طوال مراحل إنجاز هذا

المشروع. نسأل الله أن يجزيهما عنا خير الجزاء، وأن يجعل ما قدماه في ميزان حسناتهما.

## الملخص

يقدم هذا المشروع تطبيق CyberShield، وهو تطبيق ذكي للأجهزة المحمولة يهدف إلى تعزيز أمن المستخدم أثناء التعامل مع الملفات والروابط الإلكترونية. يعتمد التطبيق على تكامل عدد من الخدمات السحابية، حيث تُستخدم خدمة VirusTotal لإجراء الفحص الأمني، بينما يستفيد من تقنيات الذكاء الاصطناعي لتوليد ملخصات مبسطة وشرح نتائج الفحص بلغة سهلة، مما يساعد المستخدم على فهم مستوى الخطورة واتخاذ الإجراءات المناسبة.

تم تطوير التطبيق باستخدام Flutter، مع خادم خلفي يعتمد على Cloudflare Workers وقاعدة بيانات Cloudflare D1 والتخزين المؤقت Cloudflare KV، كما استخدمت آلية JWT لتأمين عمليات المصادقة. أظهر التطبيق قدرة على تنفيذ عمليات الفحص بكفاءة، وعرض النتائج بصورة واضحة، وتوفير تجربة استخدام سهلة وأمنة تدعم رفع مستوى الوعي بالأمن السيبراني.

### الكلمات المفتاحية

الأمن السيبراني، Flutter، VirusTotal، Cloudflare Workers، الذكاء الاصطناعي، فحص الروابط، فحص الملفات.

## ABSTRACT

This project presents CyberShield, a smart mobile application designed to improve user security when interacting with files and web links. The application integrates cloud-based security services by using VirusTotal for threat analysis and artificial intelligence to generate simplified security summaries, enabling users to understand scan results and make informed decisions.

The application was developed using Flutter, with a backend powered by Cloudflare Workers, Cloudflare D1, and Cloudflare KV, while JWT was adopted for secure authentication. The implemented system provides efficient security scanning, clear presentation of results, and a user-friendly experience that promotes cybersecurity awareness.

### Keywords

Cybersecurity, Flutter, VirusTotal, Cloudflare Workers, Artificial Intelligence, URL Scanning, File Scanning.

|    |                                                                       |
|----|-----------------------------------------------------------------------|
| 1  | الفصل الأول: الإطار العام للمشروع .....                               |
| 2  | 1-1 الخلفية والمشكلة البحثية (Background and Problem Statement) ..... |
| 2  | 1-1-1 الخلفية العامة .....                                            |
| 2  | 2-1-1 تحديد المشكلة البحثية (Research Problem) .....                  |
| 3  | 2-1 أهداف البحث (Research Objectives) .....                           |
| 3  | 1-2-1 الهدف العام (General Objective) .....                           |
| 4  | 2-2-1 الأهداف التفصيلية (Specific Objectives) .....                   |
| 5  | 3-1 أهمية البحث (Significance of the Study) .....                     |
| 5  | 1-3-1 الأهمية النظرية (Theoretical Significance) .....                |
| 6  | 2-3-1 الأهمية العملية (Practical Significance) .....                  |
| 6  | 4-1 نطاق البحث (Scope and Delimitations) .....                        |
| 7  | 5-1 مراجعة الدراسات السابقة (Literature Review) .....                 |
| 7  | 1-5-1 استراتيجيات كشف الروابط الخبيثة والتصيد الاحتيالي .....         |
| 8  | 2-5-1 رصد البرمجيات الخبيثة في بيئة الهواتف المحمولة .....            |
| 9  | 3-5-1 التعقيب العام وموقع مشروع "درع الأمان" (CyberShield) .....      |
| 10 | الفصل الثاني: التقنيات والأدوات المستخدمة .....                       |
| 11 | 1-2 إطار عمل Flutter .....                                            |
| 13 | 2-2 إدارة الحالة باستخدام Riverpod .....                              |

## فهرس المحتويات

|    |                                                      |
|----|------------------------------------------------------|
| 15 | 3-2 التنقل بين الشاشات باستخدام GoRouter             |
| 17 | 4-2 الاتصال بواجهات البرمجة باستخدام Dio             |
| 19 | 5-2 التخزين الآمن باستخدام Flutter Secure Storage    |
| 21 | 6-2 الخادم الخلفي باستخدام Hono و Cloudflare Workers |
| 23 | 7-2 قاعدة البيانات باستخدام Cloudflare D1            |
| 25 | 8-2 التخزين المؤقت باستخدام Cloudflare KV            |
| 27 | 9-2 التكامل مع VirusTotal API                        |
| 29 | 10-2 التكامل مع Google Gemini API                    |
| 31 | 11-2 المصادقة باستخدام JSON Web Token                |
| 33 | <b>3 الفصل الثالث: تحليل وتصميم النظام</b>           |
| 34 | 1-3 تحليل قاعدة البيانات                             |
| 35 | 1-1-3 جدول المستخدمين (Users)                        |
| 35 | 2-1-3 جدول عمليات الفحص (Scans)                      |
| 35 | 3-1-3 جدول نتائج VirusTotal (VT_Results)             |
| 36 | 14-3 جدول تحليلات الذكاء الاصطناعي (AI_Analyses)     |
| 36 | 5-1-3 جدول بيانات الملفات (File_Metadata)            |
| 36 | 6-1-3 العلاقات بين الكيانات                          |
| 37 | 2-3 تحليل حالات الاستخدام                            |

## فهرس المحتويات

|    |                                                            |
|----|------------------------------------------------------------|
| 38 | .....المستخدم (User) 1-2-3                                 |
| 39 | .....مدير النظام (Administrator) 2-2-3                     |
| 39 | .....الأنظمة الخارجية (External Systems) 3-2-3             |
| 39 | .....العلاقات بين حالات الاستخدام 4-2-3                    |
| 40 | .....تحليل مخطط التسلسل 3-3                                |
| 41 | .....بدء عملية الفحص 1-3-3                                 |
| 42 | .....تنفيذ عملية الفحص 2-3-3                               |
| 42 | .....تحليل النتائج بواسطة الذكاء الاصطناعي 3-3-3           |
| 42 | .....عرض التقرير للمستخدم 4-3-3                            |
| 43 | .....معمارية النظام 4-3                                    |
| 43 | .....طبقة المستخدم (User Layer) 1-4-3                      |
| 44 | .....طبقة التطبيق (Application Layer) 2-4-3                |
| 44 | .....طبقة الخادم الخلفي (Backend Layer) 3-4-3              |
| 44 | .....طبقة الخدمات الخارجية (External Services Layer) 4-4-3 |
| 45 | .....تدفق البيانات داخل النظام 5-4-3                       |
| 46 | <b>4 الفصل الرابع: التنفيذ العملي</b>                      |
| 47 | .....الواجهة الرئيسية للنظام 1-4                           |
| 48 | .....واجهة فحص الروابط 2-4                                 |

## فهرس المحتويات

|    |                                                         |
|----|---------------------------------------------------------|
| 49 | 3-4 شاشة نتائج فحص الرابط.....                          |
| 50 | 4-4 سجل عمليات الفحص.....                               |
| 51 | 5-4 الملف الشخصي للمستخدم.....                          |
| 52 | 6-4 واجهة الإعدادات.....                                |
| 53 | 7-4 واجهة فحص الروابط قبل الإدخال.....                  |
| 54 | <b>5 الفصل الخامس: الخاتمة والآفاق المستقبلية</b> ..... |
| 55 | 1-5 الخاتمة.....                                        |
| 55 | 2-5 أهم النتائج.....                                    |
| 56 | 3-5 الآفاق المستقبلية.....                              |

|                                                                                                                                       |    |
|---------------------------------------------------------------------------------------------------------------------------------------|----|
| الشكل 1-2: البنية العامة لإطار عمل Flutter وآلية بناء واجهات المستخدم باستخدام نظام الـ Widgets                                       | 11 |
| الشكل 2-2: البنية العامة لآلية إدارة الحالة باستخدام Riverpod في تطبيقات Flutter                                                      | 14 |
| الشكل 3-2: البنية العامة لآلية التوجيه وإدارة مسارات التنقل باستخدام GoRouter في تطبيقات Flutter                                      | 16 |
| الشكل 4-2: الهيكلية المعمارية لآلية الاتصال وتدفق البيانات بين تطبيق Flutter ومخدمات واجهات برمجة التطبيقات (APIs) باستخدام مكتبة Dio | 18 |
| الشكل 5-2: آلية عمل Flutter Secure Storage وتكاملها مع خدمات الحماية الأصلية في نظامي Android و iOS                                   | 20 |
| الشكل 6-2: البنية المعمارية للخادم الخلفي باستخدام إطار عمل Hono و Cloudflare Workers                                                 | 22 |
| الشكل 7-2: تكامل الخادم الخلفي Cloudflare Workers مع قاعدة البيانات السحابية Cloudflare D1                                            | 24 |
| الشكل 8-2: تكامل الخادم الخلفي Cloudflare Workers مع خدمة التخزين المؤقت Cloudflare KV                                                | 26 |
| الشكل 9-2: آلية فحص الملفات والروابط والتكامل مع واجهة VirusTotal API                                                                 | 28 |
| الشكل 10-2: دورة المصادقة وتدفق الرموز المميزة (JWT) بين تطبيق العميل والخادم                                                         | 32 |
| الشكل 1-3: مخطط الكيانات والعلاقات (Entity Relationship Diagram) لقاعدة بيانات نظام CyberShield                                       | 34 |
| الشكل 2-3: مخطط حالات الاستخدام (Use Case Diagram) لنظام CyberShield                                                                  | 38 |

|                                                                          |    |
|--------------------------------------------------------------------------|----|
| الشكل 3-3: مخطط التسلسل (Sequence Diagram) لعملية الفحص والتحليل في نظام |    |
| CyberShield.....                                                         | 41 |
| الشكل 4-3: المعمارية العامة لنظام CyberShield.....                       | 43 |
| الشكل 1-4: الواجهة الرئيسية لتطبيق CyberShield.....                      | 47 |
| الشكل 2-4: واجهة إدخال الرابط وإعدادات الفحص.....                        | 48 |
| الشكل 3-4: نتيجة فحص الرابط والتقدير الأمني المبسط.....                  | 49 |
| الشكل 4-4: واجهة سجل الفحوصات السابقة.....                               | 50 |
| الشكل 5-4: واجهة الملف الشخصي وإحصائيات الاستخدام.....                   | 51 |
| الشكل 6-4: واجهة إعدادات التطبيق.....                                    | 52 |
| الشكل 7-4: واجهة فحص الروابط قبل إدخال عنوان إلكتروني.....               | 53 |

## فهرس الجداول

|                                                                             |    |
|-----------------------------------------------------------------------------|----|
| الجدول 1-2: مقارنة بين Flutter وبعض أطر تطوير التطبيقات متعددة المنصات..... | 12 |
| الجدول 2-2: أنواع المزودات الرئيسة في Riverpod واستخداماتها الوظيفية.....   | 15 |
| الجدول 3-2: أهم خصائص GoRouter في تطبيقات Flutter.....                      | 17 |
| الجدول 4-2: مقارنة بين Dio ومكتبة HTTP القياسية في Flutter.....             | 19 |
| الجدول 5-2: مقارنة بين SharedPreferences وFlutter Secure Storage.....       | 21 |
| الجدول 6-2: لتوزيع الوظيفي لمكونات الخادم الخلفي في مشروع CyberShield.....  | 23 |
| الجدول 7-2: الخصائص الرئيسة لقاعدة البيانات Cloudflare D1.....              | 25 |
| الجدول 8-2: مقارنة بين Cloudflare KV وCloudflare D1.....                    | 27 |
| الجدول 9-2: أبرز خدمات VirusTotal API.....                                  | 29 |
| الجدول 10-2: مراحل عمل Google Gemini API في مشروع CyberShield.....          | 30 |
| الجدول 11-2: المكونات البنيوية لرمز JSON Web Token (JWT).....               | 32 |

# الفصل الأول: الإطار العام

## للمشروع

## 1-1 الخلفية والمشكلة البحثية (Background and Problem Statement)

### 1-1-1 الخلفية العامة

يشهد العالم تسارعاً غير مسبوق في التحول الرقمي، حيث أصبحت الهواتف الذكية المنصة الأساسية للأنشطة اليومية بدءاً من المعاملات المالية وصولاً إلى التواصل الاجتماعي [1][2]. وقد بلغ عدد مستخدمي الهواتف الذكية عالمياً أكثر من 6.5 مليار مستخدم في عام 2025، مما جعلها هدفاً استراتيجياً للهجمات السيبرانية [2].

وتشير الدراسات الحديثة إلى تزايد مطرد في الهجمات السيبرانية الموجهة للأجهزة المحمولة، خاصة برمجيات الفدية (Ransomware) والتصيد الاحتيالي (Phishing) [3][9]. ففي عام 2024 وحده، سجلت شركات الأمن السيبراني زيادة بنسبة 45% في محاولات الاختراق عبر تطبيقات Android الملغمة مقارنة بالعام السابق [4].

### 2-1-1 تحديد المشكلة البحثية (Research Problem)

بالرغم من تطور حلول الأمن السيبراني، تظهر ثلاث فجوات جوهرية في الأدبيات الحديثة:

#### الفجوة الأولى: محدودية المحركات الفردية

تعتمد التطبيقات الأمنية التقليدية على محرك كشف واحد، مما يحد من دقة الكشف خاصة في مواجهة هجمات اليوم الصفر (Zero-day Attacks) [5]. فقد أظهرت الدراسات أن الأنظمة القائمة على محرك واحد تحقق دقة لا تتجاوز 84.99% في ظروف التشغيل العادية، مع معدلات إنذارات كاذبة مرتفعة [6].

#### الفجوة الثانية: المفاضلة بين الدقة والكفاءة

الحلول متعددة المحركات التي تدمج عدة تقنيات مثل XGBoost و Google Safe Browsing تحقق دقة محسنة، لكنها تستهلك موارد باهظة من البطارية والمعالج [7]. كما أن نماذج التعلم العميق مثل BERT، رغم دقتها في فهم الدلالات العميقة للروابط، تتطلب موارد حسابية عالية تستنزف البطارية وترفع زمن الاستجابة (Latency) بشكل كبير [8].

### الفجوة الثالثة: الفجوة المعرفية في التقارير الأمنية

تقدم الأنظمة الحالية تقارير تقنية معقدة لا تفهمها الفئات غير التقنية من المستخدمين، مما يحد من فعالية التدخلات الوقائية [9]. وأظهرت دراسة حديثة أن 78% من المستخدمين العاديين لا يستطيعون تفسير التحذيرات الأمنية التقليدية بشكل صحيح [10].

## 2-1 أهداف البحث (Research Objectives)

### 1-2-1 الهدف العام (General Objective)

تطوير إطار عمل هجين (Hybrid Framework) لتطبيق أمني ذكي للهواتف المحمولة يحقق التوازن الأمثل بين ثلاثة أبعاد رئيسية:

🛡️ دقة الكشف المتعدد المحركات عبر التكامل مع منصات الأمن السحابية،

🔒 كفاءة استهلاك الموارد المحلية عبر خوارزميات التجزئة (Hashing) والمعالجة على الجهاز

،(On-Device Processing)

🧠 تجربة مستخدم مبسطة وفعالة عبر وكيل ذكاء اصطناعي توليدي (Generative AI Agent)

يعتمد على هندسة التلقين المتقدمة (Advanced Prompt Engineering).

## 2-2-1 الأهداف التفصيلية (Specific Objectives)

- 🛡️ تقليل استهلاك بيانات الإنترنت عبر استخدام خوارزمية SHA-256 لتوليد بصمة رقمية (Hash) محلياً بدلاً من رفع الملفات الكاملة للسحابة، مع الحفاظ على خصوصية المستخدم من خلال عدم إرسال البيانات الحساسة.
- 🛡️ تحقيق توازن فعال بين الأداء والدقة عبر اعتماد آلية ذكية تجمع بين المعالجة المحلية والمعالجة السحابية، مع ضمان عمل التطبيق بكفاءة على الأجهزة ذات الموارد المحدودة.
- 🛡️ تطوير نظام فحص متكامل يعتمد على الفحص المتعدد المحركات ( Multi-Engine Scanning ) لرفع دقة الكشف وتقليل الإنذارات الكاذبة.
- 🛡️ تطبيق تقنيات التعلم العميق لتحليل الروابط (URL Analysis) والكشف عن محاولات التصيد الاحتيالي والمواقع الضارة.
- 🛡️ دعم التحديث الفوري لقواعد البيانات الأمنية لضمان الكشف عن أحدث التهديدات السيبرانية.
- 🛡️ بناء وكيل ذكاء اصطناعي قادر على تبسيط التقارير التقنية وتحويلها إلى معلومات مفهومة للمستخدم غير المختص.
- 🛡️ توظيف تقنيات هندسة التلقين المتقدمة (Advanced Prompt Engineering) لتحسين جودة تحليل البيانات الأمنية وتقديم نتائج دقيقة وواضحة.
- 🛡️ تقديم توجيهات وقائية فورية للمستخدم بناءً على مستوى الخطورة المكتشف.
- 🛡️ تحسين قابلية الفهم لدى المستخدمين غير التقنيين، مع دعم تعدد اللغات (العربية والإنجليزية).

👤 تحقيق زمن استجابة منخفض مع الحفاظ على كفاءة استهلاك الطاقة وعدم استنزاف البطارية.

👤 اختبار النظام في بيئات تشغيل مختلفة (شبكات متعددة وحالات اتصال متنوعة) لضمان الموثوقية.

👤 إجراء مقارنات معيارية مع الأنظمة القائمة لتقييم الأداء من حيث الدقة والكفاءة.

👤 اختبار قابلية التوسع لضمان استقرار النظام عند معالجة عدد كبير من الطلبات المتزامنة.

👤 جمع البيانات الميدانية وتحليل سلوك المستخدمين لتحسين تجربة الاستخدام في الإصدارات المستقبلية.

## 3-1 أهمية البحث (Significance of the Study)

### 1-3-1 الأهمية النظرية (Theoretical Significance)

يسهم هذا البحث في إثراء الأدبيات العلمية من خلال المساهمات النظرية التالية:

👤 تقديم نموذج هجين جديد يجمع بين خوارزميات التجزئة (SHA-256) والتحليل السحابي

المتعدد المحركات، مما يعالج القيود المنهجية في الأنظمة القائمة التي تعتمد على نهج واحد

فقط، حيث أظهرت الدراسات الحديثة أن الأطر الهجينة التي تدمج بين التحليل الثابت

والديناميكي تحقق دقة أعلى في كشف البرمجيات الخبيثة مقارنة بالنهج المنفردة [5][7].

👤 تطوير إطار عمل يوازن بين المعالجة على الجهاز (On-Device AI) والمعالجة السحابية،

معالجاً القيود المرتبطة بموارد الأجهزة المحمولة، حيث أثبتت الأبحاث الحديثة إمكانية تطبيق

نماذج تعلم عميق فعالة ضمن بيئات محدودة الموارد [4][8].

إثراء النظرية في مجال تحسين السمات (Feature Optimization) من خلال تقليل أبعاد البيانات دون التأثير على دقة النماذج، مما يساهم في تطوير منهجيات التعامل مع البيانات عالية الأبعاد في الأمن السيبراني [4].

تقديم مساهمة في مجال هندسة التلقين (Prompt Engineering) لتبسيط التقارير الأمنية المعقدة، مما يفتح آفاقاً جديدة لدمج الذكاء الاصطناعي التوليدي في الأمن السيبراني [9].

### 1-3-2 الأهمية العملية (Practical Significance)

توفير أداة أمنية متكاملة للمستخدمين النهائيين في بيئات الموارد المحدودة، مع تحقيق كفاءة عالية في استهلاك الموارد [6].

تقليل مخاطر الاختراق من خلال دمج محركات كشف متعددة، مما يحسن القدرة على اكتشاف التهديدات المعقدة [7].

تحسين الوعي الأمني لدى المستخدمين عبر تقديم تقارير مبسطة مدعومة بالذكاء الاصطناعي [10].

دعم تطوير بنية تحتية رقمية آمنة ومبتكرة، بما يتماشى مع متطلبات الأنظمة الحديثة [1].

الحفاظ على الخصوصية من خلال المعالجة المحلية للبيانات الحساسة بدلاً من الاعتماد الكامل على السحابة [5].

### 1-4 نطاق البحث (Scope and Delimitations)

يقتصر نطاق هذا البحث على تطوير تطبيق أمني ذكي للهواتف المحمولة خلال العام الدراسي 2025-2026، بحيث يشمل مراحل التصميم والتطوير والاختبار والتقييم، مع استهداف زمن استجابة منخفض لا يتجاوز 3 ثوانٍ للاستعلامات المتكررة. يعتمد التطبيق على إطار Flutter للعمل على نظام Android فقط دون دعم iOS، ويستهدف الأجهزة ذات المواصفات المتوسطة مع ضمان كفاءة

استهلاك الموارد. يركز النظام على التكامل مع خدمات سحابية مثل VirusTotal API للفحص الأمني وGoogle Gemini API لمعالجة النتائج، مع التركيز على فحص الملفات باستخدام خوارزمية SHA-256 وفحص الروابط عبر واجهات REST، بالإضافة إلى استخدام وكيل ذكاء اصطناعي لتبسيط التقارير. ويقتصر العمل على التحليل الثابت دون تنفيذ التحليل الديناميكي أو مراقبة سلوك الملفات أو تحليل حركة الشبكة، كما لا يشمل تطوير خوادم خلفية خاصة أو أنظمة حماية فورية، ويتم تقييم النظام ضمن بيئة محلية محدودة دون التوسع الجغرافي في هذه المرحلة.

## 5-1 مراجعة الدراسات السابقة (Literature Review)

لم تعد الهجمات السيبرانية كما كانت. ففي السنوات الأخيرة، شهدنا قفزة نوعية في تعقيدها، خاصة تلك التي تستهدف هواتفنا الذكية عبر التصيد الاحتيالي والبرمجيات الخبيثة. وقد استجاب الباحثون لهذا التحدي بتوجه واضح نحو تقنيات التعلم الآلي والتحليل الديناميكي كخط دفاع متقدم. في هذا القسم، نستعرض أبرز ما توصلت إليه الدراسات الحديثة (2022-2025)، ونسلط الضوء على المنهجيات القائمة، ثم نحدد الفجوات التي جاء مشروع "درع الأمان" (CyberShield) ليملأها.

### 1-5-1 استراتيجيات كشف الروابط الخبيثة والتصيد الاحتيالي

🛡️ **دمج الخوارزميات مع الواجهات البرمجية:** Kumar & Sharma (2025) قدموا تطبيقًا يجمع بين خوارزمية XGBoost لتحليل بنية الروابط وقواعد بيانات Google Safe Browsing. النتيجة كانت نجاح النموذج في تقليل الإنذارات الكاذبة، لكنه بقي أداة تقنية صرفة تفتقر إلى شرح مفهوم للمستخدم العادي [6].

🛡️ **التعلم العميق وتحليل السياق:** Hassan et al. (2025) استخدموا نموذج BERT لفهم الدلالات العميقة للروابط، مما مكّنهم من اكتشاف تهديدات تتجاوز القوائم السوداء التقليدية. لكن المشكلة كانت في "العبء الحسابي"—تشغيل هذه النماذج محليًا على الهواتف يستنزف البطارية

ويرفع زمن الاستجابة (Latency)، مما يحد من جدواها في الاستخدام اليومي [8]. من جهة أخرى، أعلن فريق Google Chrome Security (2025) عن استخدام تقنيات Gemini Nano للكشف عن الاحتيال مباشرة على الجهاز، مع تحسين استهلاك الموارد عبر التحكم الذكي في GPU [9].

### 1-5-2 رصد البرمجيات الخبيثة في بيئة الهواتف المحمولة

تنوعت المقاربات البحثية في تحليل أمن التطبيقات بين التحليل الثابت والتحسين المعماري:

 **التحليل الثابت واستخراج السمات:** Alam & Vuong (2024) اقترحوا إطار عمل يفكك التطبيقات محليًا لاستخراج الأذونات ونداءات الواجهات البرمجية (API Calls)، ثم يصنفها عبر Random Forest بدقة بلغت 99.03% على مجموعة Drebin و100% على CCCS-CIC-AndMal-2020 [2]. من جانب آخر، ركز Akbar et al (2022) على فحص الأذونات كخط دفاع أولي بسيط، محققين دقة 89.96% باستخدام Random Forest على 10,000 تطبيق [1].

 **تخصيص الموارد والموثوقية:** Alam & Vuong (2024) أكدوا على أهمية تقليص الميزات (Feature Reduction) باستخدام Information Gain وGini Index لتسريع أداء Random Forest، مما قلل أبعاد البيانات بنسبة 77% دون التضحية بالدقة [2]. كما سلطت الدراسة الضوء على VirusTotal كمرجع موثوق (Ground Truth) لتصنيف البيانات، مما يجعله معيارًا ذهبيًا في التقييم الأمني [8].

 **هندسة التعلم العميق:** في 2024، طرح Seyfari & Meimandi أطرًا تعتمد على mini-ResNet مع طبقات انتباه (Attention Layers) لتحليل صور مشابهة لرمز QR بناءً على معلومات الأذونات، موازنين بين الدقة العالية والموارد المحدودة [10].

### 3-5-1 التعقيب العام وموقع مشروع "درع الأمان" (CyberShield)

بنظرة نقدية على الدراسات السابقة، يتضح أن الحلول القائمة غالبًا ما تقع في فخ المفاضلة بين كفاءة الأداء ودقة التحليل، ناهيك عن الفجوة الواضحة في تجربة المستخدم [2][6]. هنا بالضبط يأتي دور مشروع "درع الأمان"، الذي يتبنى معمارية هجينة تقوم على ثلاثة محاور:

🛡️ **التشخيص المتعدد والموثق:** بدلاً من الاكتفاء بفحص سطحي للأذونات [3][11]، يتكامل المشروع مع VirusTotal عبر API، مما يتيح الاستعانة بعشرات المحركات الأمنية العالمية وفحص البنية الداخلية للملفات بدقة لا يمكن تحقيقها محليًا [8].

🛡️ **هندسة استهلاك الموارد (Optimization):** لمواجهة مشكلة استهلاك النطاق الترددي وزمن التأخير، يستخدم المشروع SHA-256 لتوليد "بصمة رقمية" (Hash) فريدة محليًا. بدلاً من رفع الملفات الضخمة، يُرسل الـ Hash فقط—هذا هو التحسين الأمثل في هندسة الاتصالات [2][5].

🛡️ **تبسيط المخرجات عبر الوكيل الذكي (AI Agent):** يعالج المشروع "الفجوة المعرفية" لدى المستخدمين بدمج وكيل ذكاء اصطناعي يعتمد على هندسة التلقين ( Prompt Engineering). يحول الوكيل البيانات المعقدة إلى تقارير مفهومة توضح طبيعة الخطر وتقدم نصائح فورية، محوّلًا التطبيق من أداة تقنية صماء إلى مستشار أمني تفاعلي [6][9].

"درع الأمان" ليس مجرد تطبيق حماية، بل توليفة هندسية ذكية تجمع بين دقة الحوسبة السحابية، وكفاءة المعالجة المحلية، ومرونة الذكاء الاصطناعي. يضع المشروع معيارًا جديدًا لتطبيقات الأمن السيبراني المبنية على Flutter، متجاوزًا القيود التي عرقلت الأبحاث السابقة [2][6][8].

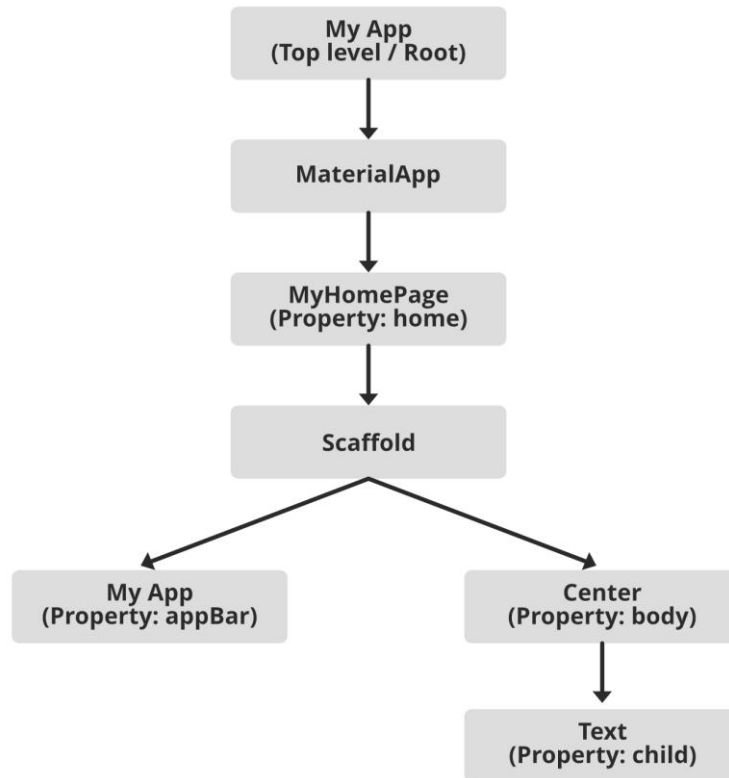
الفصل الثاني: التقنيات

والأدوات المستخدمة

## 1-2 إطار عمل Flutter

يُعدّ Flutter إطار عمل مفتوح المصدر طوّره شركة Google لتطوير التطبيقات متعددة المنصات بالاعتماد على لغة البرمجة Dart. يتيح هذا الإطار بناء تطبيقات متكاملة تعمل على أنظمة Android و iOS والويب وسطح المكتب باستخدام قاعدة برمجية موحدة (Single Codebase)، ويتميز باحتوائه على مكتبة غنية من عناصر واجهة المستخدم (Widgets) التي تُسهّم في تصميم واجهات تفاعلية ذات أداء عالٍ وتجربة مستخدم متجانسة. علاوة على ذلك، يعتمد Flutter على محرك رندرة (رسم) خاص به لعرض العناصر مباشرة دون الحاجة إلى المكونات الأصلية للنظام، مما يعزز سرعة الأداء واستقرار التطبيق البرمجي [11].

ويمكن تمثيل البنية الهيكلية لآلية بناء الواجهات في هذا الإطار من خلال الشكل (1-2).



الشكل 1-2: البنية العامة لإطار عمل Flutter وآلية بناء واجهات المستخدم باستخدام نظام ال Widgets.

وفي سياق مشروع CyberShield، جرى اعتماد إطار العمل Flutter لتطوير تطبيق الهاتف المحمول الخاص بالنظام، حيث صُممت واجهات المستخدم بناءً على بنية هرمية من الشاشات (Screens) والوحدات المشتركة (Widgets). ولضمان كفاءة الصيانة وقابلية التوسع، تم تنظيم البنية البرمجية للمشروع ضمن مكونات رئيسية تشمل: الطبقة الأساسية (Core)، ونماذج البيانات (Models)، والخدمات (Services)، وإدارة الحالة (Providers). كما تم دمج مجموعة من الحزم البرمجية المتخصصة لتسهيل إدارة الحالة، والتنقل بين الشاشات، والتخزين الآمن، والتكامل مع واجهات برمجة التطبيقات (APIs)، بما يلبي المتطلبات الوظيفية والأمنية للنظام [12].

ويوضح الجدول (1-2) مقارنة تحليلية بين Flutter وأبرز أطر التطوير متعددة المنصات المنافسة من حيث لغة البرمجة، وآلية بناء الواجهات، والخصائص التشغيلية.

الجدول 1-2: مقارنة بين Flutter وبعض أطر تطوير التطبيقات متعددة المنصات

| المعيار           | Flutter                    | React Native                     | NET MAUI.                          |
|-------------------|----------------------------|----------------------------------|------------------------------------|
| لغة البرمجة       | Dart                       | JavaScript                       | #C                                 |
| قاعدة الشيفرة     | موحدة (واحدة)              | موحدة (واحدة)                    | موحدة (واحدة)                      |
| آلية بناء الواجهة | عناصر مخصصة (Widgets)      | مكونات أصلية (Native Components) | عناصر تحكم أصلية (Native Controls) |
| الأداء            | مرتفع                      | مرتفع                            | مرتفع                              |
| دعم المنصات       | Android, iOS, Web, Desktop | Android, iOS                     | Android, iOS, Windows, macOS       |

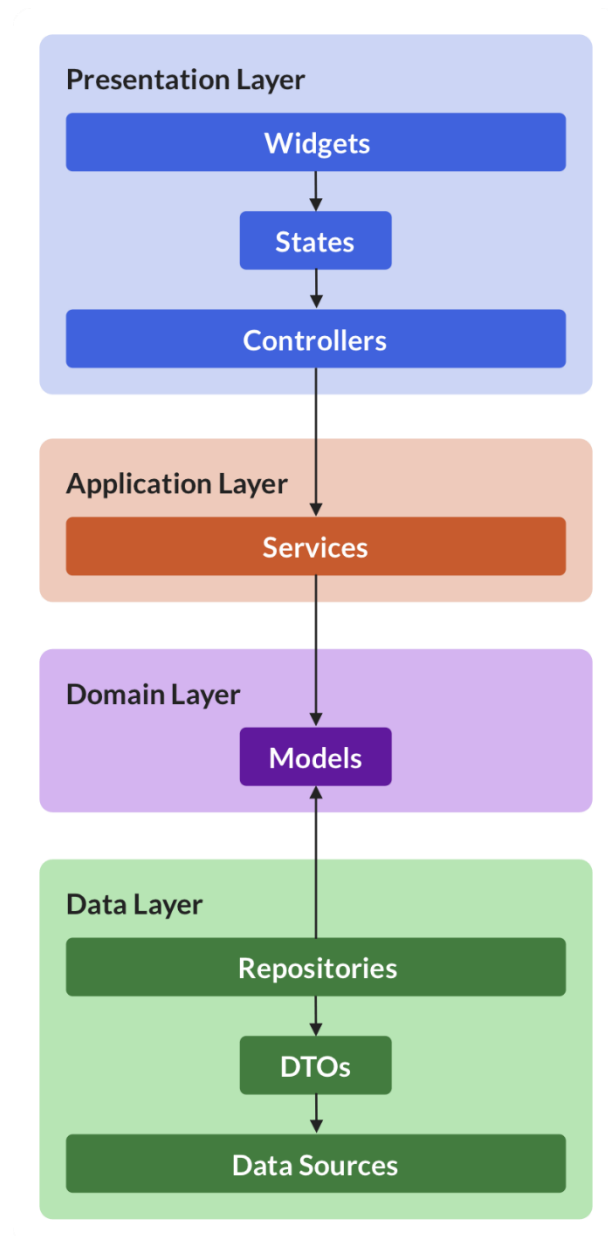
(مقتبس من المرجع [11]).

## 2-2 إدارة الحالة باستخدام Riverpod

تُعد إدارة الحالة (State Management) إحدى الركائز الهندسيّة الأساسية في تطوير تطبيقات Flutter؛ إذ تهدف إلى تنظيم وتوجيه البيانات المشتركة بين مختلف مكونات التطبيق، وضمان مزامنة واجهة المستخدم لحظياً مع أي تغيير يطرأ على تلك البيانات. ويؤدي اختيار البنية الملائمة لإدارة الحالة إلى تحسين كفاءة الشيفرة البرمجية، وتقليل الاعتماد المتبادل (Coupling) بين المكونات، فضلاً عن تسهيل عمليات الصيانة والاختبار والتطوير المستقبلي للمنظومة البرمجية [13].

وفي إطار تطوير مشروع CyberShield، جرى اعتماد حزمة Riverpod كأداة رئيسية لإدارة الحالة والتحكم في تدفق البيانات داخل التطبيق. وتتولى هذه الحزمة تنظيم العمليات الحيوية المتبادلة بين واجهات النظام المختلفة، مثل: إجراءات المصادقة وتأمين تسجيل الدخول، وتنفيذ عمليات الفحص الأمني، وعرض سجل العمليات. وينعكس هذا الاعتماد بوضوح في الهيكل التنظيمي للمشروع عبر تخصيص طبقة مستقلة للمزودات (Providers)، تتضمن المزودات الأساسية المعنية بالتحقق والفحص، مما يضمن الفصل التام بين منطق العمل البرمجي (Business Logic) وعناصر واجهة المستخدم (UI). علاوة على ذلك، تتميز Riverpod بآلية تحديث انتقائية تقتصر على إعادة بناء (Rebuild) الأجزاء المتأثرة مباشرة بتغير الحالة، مما يرفع كفاءة الأداء التشغيلي ويحد من استهلاك موارد الجهاز في عمليات إعادة الرسم غير الضرورية للواجهات [14].

ويوضح الشكل (2-2) المفهوم الهيكلي لآلية تدفق البيانات وإدارة الحالة باستخدام Riverpod، والعلاقة التفاعلية بين مزودات البيانات وواجهات المستخدم.



الشكل 2-2: البنية العامة لآلية إدارة الحالة باستخدام Riverpod في تطبيقات Flutter.

ويبين الجدول (2-2) تصنيفاً تحليلياً لأهم أنواع المزودات (Providers) التي توفرها Riverpod، موضحاً الاستخدام الوظيفي الأمثل لكل نوع بناءً على طبيعة البيانات والعمليات البرمجية داخل النظام.

الجدول 2-2: أنواع المزودات الرئيسية في Riverpod واستخداماتها الوظيفية

| نوع المزود (Provider Type) | الاستخدام والتوظيف البرمجي (Use Case)                                                        |
|----------------------------|----------------------------------------------------------------------------------------------|
| <b>Provider</b>            | توفير البيانات الثابتة أو الكائنات المشتركة التي لا تتغير قيمتها ديناميكياً.                 |
| <b>StateProvider</b>       | إدارة وتحديث الحالات المتغيرة البسيطة (مثل المتغيرات المنطقية أو الأرقام).                   |
| <b>FutureProvider</b>      | معالجة العمليات غير المتزامنة (Asynchronous) وجلب البيانات من واجهات برمجة التطبيقات (APIs). |
| <b>StreamProvider</b>      | التعامل مع تدفقات البيانات المستمرة واللحظية (Real-time Data Streams).                       |
| <b>NotifierProvider</b>    | إدارة الحالات المعقدة وعزل منطق الأعمال المتطور عن الواجهة الرسومية بالكامل.                 |

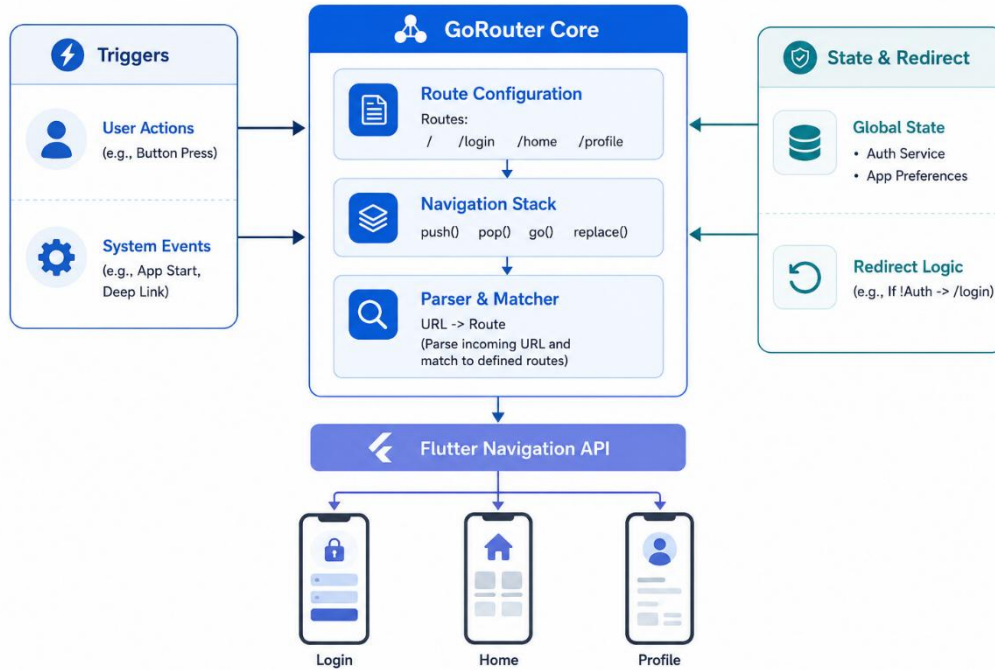
(مقتبس من المرجع [14]).

## 3-2 التنقل بين الشاشات باستخدام GoRouter

تُعدّ إدارة التنقل بين واجهات التطبيق (Navigation Management) عنصراً مفصلياً في هندسة تطبيقات الهواتف المحمولة؛ إذ تضمن سلسلة انتقال المستخدم وتحافظ على التسلسل المنطقي لسير العمل. وفي هذا السياق، تُقدم حزمة GoRouter معمارية حديثة ومتقدمة لإدارة التوجيه في بيئة Flutter، معتمدةً على أسلوب التعريف المركزي للمسارات (Declarative Routing). يتيح هذا النهج تحكماً أدق في الروابط العميقة (Deep Linking) ومعالجة فعالة لعمليات إعادة التوجيه (Redirection) بناءً على التغيرات اللحظية في حالة المستخدم أو حالة التطبيق [15].

وفيما يخص الهيكلية البرمجية لمشروع CyberShield، تم توظيف مكتبة GoRouter لبناء نظام توجيه مركزي ومرن عبر ملف `app_router.dart`. تبدأ رحلة المستخدم من الشاشة الترحيبية (Splash Screen)، ليعمل النظام على توجيهه تلقائياً إما إلى شاشة تسجيل الدخول أو الواجهة

الرئيسية استناداً إلى حالة المصادقة الفعالة (Authentication State). وتمتد هذه الإدارة لتشمل كافة التفرعات الوظيفية، كالانتقال بين شاشات فحص الروابط والملفات، وسجل العمليات، والملف الشخصي، وصولاً إلى شاشات عرض التقارير الأمنية التفصيلية. علاوة على ذلك، تلعب آلية إعادة التوجيه دوراً أمنياً بارزاً في النظام، حيث تتحقق من صلاحيات الوصول قبل تحميل الشاشات، مما يمنع الوصول غير المصرح به للأجزاء الحساسة من التطبيق ويجعل عملية التنقل أكثر أماناً وتنظيماً [16]. ويوضح الشكل (3-2) البنية العامة لآلية إدارة التنقل المركزية وتدفق الواجهات باستخدام GoRouter وارتباطها بحالة المصادقة.



الشكل 3-2: البنية العامة لآلية التوجيه وإدارة مسارات التنقل باستخدام GoRouter في تطبيقات Flutter.

ويُلخّص الجدول (3-2) أبرز الخصائص التقنية والوظيفية التي تميز مكتبة GoRouter في إدارة مسارات التطبيق بكفاءة.

الجدول 2-3: أهم خصائص GoRouter في تطبيقات Flutter.

| الخاصية (Feature)                     | الوصف التقني والوظيفي                                                             |
|---------------------------------------|-----------------------------------------------------------------------------------|
| التعريف المركزي (Declarative Routing) | بناء هيكل موحد يضم جميع مسارات التطبيق في نقطة تحكم واحدة ومقروءة.                |
| الروابط العميقة (Deep Linking)        | إتاحة الوصول المباشر إلى شاشات محددة داخل التطبيق عبر روابط خارجية.               |
| إعادة التوجيه (Redirection)           | توجيه مسار المستخدم ديناميكياً استناداً إلى الشروط (مثل التحقق من حالة المصادقة). |
| التنقل المتداخل (Nested Navigation)   | إدارة التنقل ضمن الشاشات الفرعية المتداخلة بمرونة (مثل أشرطة التنقل السفلية).     |
| مزامنة المسارات (URL Synchronization) | المزامنة التلقائية للمسارات مع عنوان الويب (URL) في منصات التشغيل المتعددة.       |

(مقتبس من المرجع [16]).

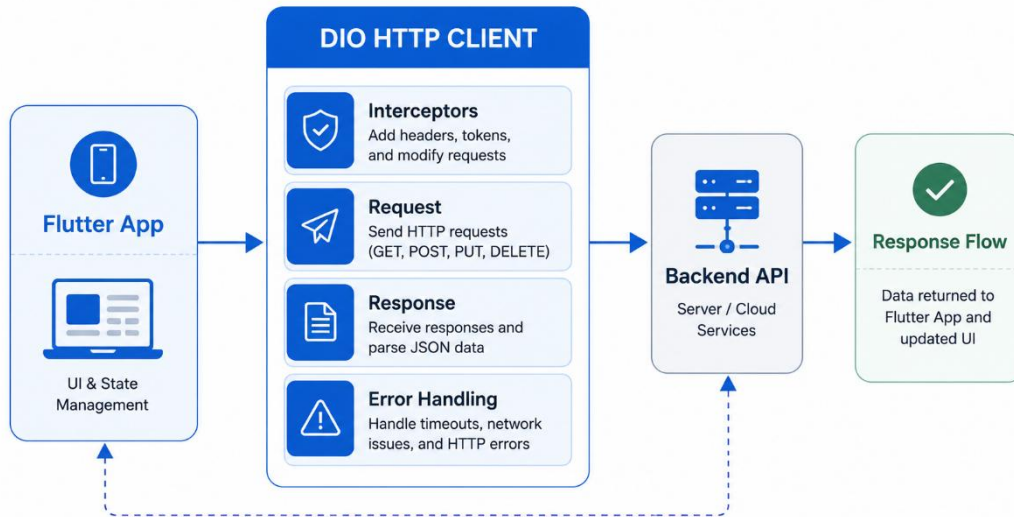
## 4-2 الاتصال بواجهات البرمجة باستخدام Dio

تُعدّ واجهات برمجة التطبيقات (APIs) العصب المحرّك لتبادل البيانات بين تطبيقات الهواتف المحمولة والخدمات السحابية الخلفية (Backend). وفي بيئة Flutter، تتعدد المكتبات المخصصة لمعالجة طلبات بروتوكول نقل النص الفائق (HTTP)، إلا أن مكتبة Dio تبرز كخيار هندسي متقدم بفضل ما توفره من إمكانيات شاملة؛ كالإدارة الدقيقة للمهلات الزمنية (Timeouts)، والتحكم بترويسات الطلبات (Headers)، فضلاً عن دعمها المدمج لآلية الاعتراضات (Interceptors) التي تلعب دوراً محورياً في مراقبة وتأمين تدفق العمليات الشبكية [17].

وفي سياق البنية المعمارية لمشروع CyberShield، شكّلت مكتبة Dio حجر الأساس لتنفيذ كافة قنوات الاتصال المشفرة بين التطبيق ومخدمات النظام. حيث وُظّفت لإدارة العمليات الحيوية بدءاً من المصادقة (تسجيل الدخول وإنشاء الحساب)، مروراً بإرسال حزم البيانات الدقيقة لفحص الملفات

والروابط، وصولاً إلى استرجاع السجلات الأمنية وإدارة بيانات المستخدم. وتتم معالجة هذه الطلبات عبر إرسال واستقبال البيانات بصيغة JSON، ومن ثم تحويلها إلى نماذج بيانات (Models) مهيكلية تُغذي واجهات واجهة المستخدم. وقد أسهم العزل البرمجي لهذه العمليات وحصرها ضمن طبقة الخدمات (Services) في تعزيز قابلية صيانة النظام وتوسيع وظائفه مستقبلاً.

ويوضح الشكل (4-2) المخطط الهيكلي لدورة حياة الاتصال، بدءاً من إرسال الطلبات وتمثيلها عبر طبقة الاعتراضات (Interceptors)، وصولاً إلى معالجة الاستجابات الواردة من الخادم الخلفي والتعامل مع الأخطاء المحتملة.



الشكل 4-2: الهيكلية المعمارية لآلية الاتصال وتدفق البيانات بين تطبيق Flutter ومخدمات واجهات برمجة التطبيقات (APIs) باستخدام مكتبة Dio.

علاوة على ما سبق، توفر Dio نظاماً مركزياً متيناً لمعالجة استثناءات وأخطاء الشبكة، كحالات تجاوز المهلة الزمنية، أو انقطاع الاتصال، أو فشل استجابة المخدم. وتتيح هذه الاستراتيجية للنظام التقاط الأخطاء برمجياً قبل تأثيرها على الواجهة، وترجمتها إلى رسائل تنبيهية واضحة للمستخدم، مما يضمن استقرار التطبيق ويمنع التوقفات المفاجئة [18].

ويُبيّن الجدول (4-2) مقارنة تقنية تبرز تفوق مكتبة Dio على مكتبة HTTP التقليدية في تلبية متطلبات بناء التطبيقات المعقدة.

الجدول 4-2: مقارنة بين Dio ومكتبة HTTP القياسية في Flutter

| المعيار الفني                    | مكتبة Dio                   | مكتبة HTTP القياسية         |
|----------------------------------|-----------------------------|-----------------------------|
| إدارة المهلات الزمنية (Timeouts) | مدعومة                      | مدعومة                      |
| الاعتراضات (Interceptors)        | مدعومة بالكامل              | غير مدعومة                  |
| آلية معالجة الأخطاء              | متقدمة (تصنيف دقيق للأخطاء) | أساسية                      |
| رفع وتنزيل الملفات (Multipart)   | مدعومة بكفاءة عالية         | مدعومة                      |
| تحويل البيانات (JSON Parsing)    | مدمج وآلي                   | مدعوم (يتطلب تدخلاً يدوياً) |
| الإدارة المركزية للطلبات         | مدعومة (عبر BaseOptions)    | محدودة                      |

(مقتبس من المرجع [17]).

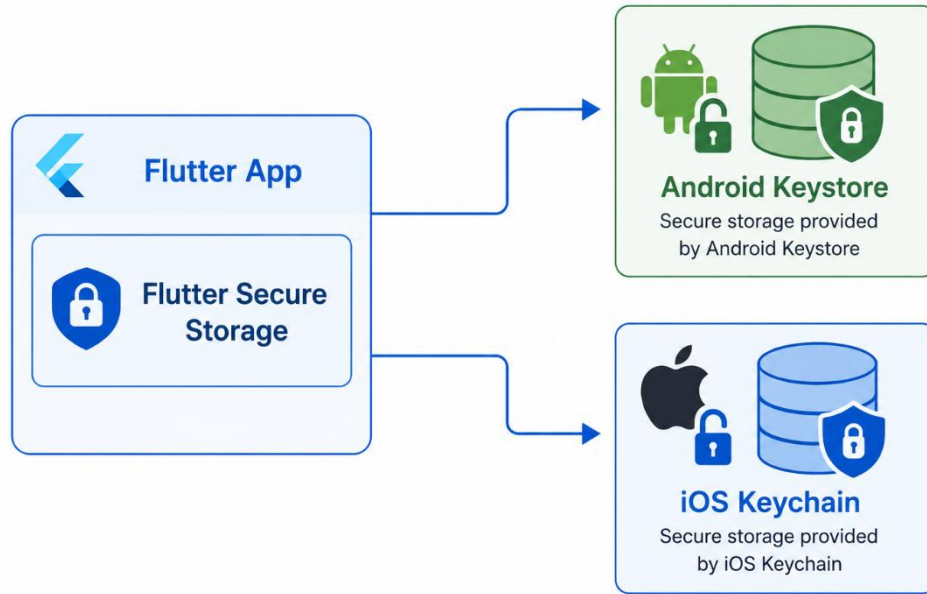
## 5-2 التخزين الآمن باستخدام Flutter Secure Storage

تُعَدّ حماية البيانات الحساسة ركيزة أساسية في تصميم تطبيقات الهواتف المحمولة، لاسيما تلك التي تتعامل مع أنظمة المصادقة والمعلومات الشخصية الدقيقة. ولتحقيق هذا الهدف، تُوظف مكتبة Flutter Secure Storage لتخزين البيانات بشكل آمن، معتمدةً على البنى التحتية للتشفير المدمجة والمدعومة عتادياً في أنظمة التشغيل، مثل نظام (Android Keystore) في بيئة أندرويد، ونظام (Keychain) في بيئة iOS. تضمن هذه المعمارية تشفير البيانات محلياً بفعالية عالية، مما يمنع أي وصول غير مصرح به [19].

وفيما يخص البنية الأمنية لمشروع CyberShield، تم الاعتماد على هذه المكتبة لحفظ بيانات المصادقة الحيوية للمستخدم، كرموز الوصول (Access Tokens) وبيانات الجلسة (Session Data) الضرورية لاستمرار حالة تسجيل الدخول. يمنع هذا التوجه المعماري تخزين المعلومات الحساسة كنصوص واضحة داخل ملفات التخزين القياسية، مما يرفع من موثوقية التطبيق ويحافظ على سرية

بيانات المستخدم أثناء دورة حياة النظام. علاوة على ذلك، تُستثمر هذه التقنية في الاسترداد التلقائي لبيانات الجلسة عند إقلاع التطبيق، مما يوفر تجربة مستخدم (UX) سلسلة ومرنة دون المساومة على المعايير الأمنية. وقد تم هيكلة هذه العمليات برمجياً وعزلها داخل طبقة الخدمات (Services)، لضمان استقلالية إدارة بيانات المصادقة عن واجهة المستخدم.

ويوضح الشكل (5-2) الآلية العامة لتخزين البيانات الحساسة وارتباطها المباشر بخدمات الحماية الأصلية التي يوفرها نظام التشغيل.



الشكل 5-2: آلية عمل Flutter Secure Storage وتكاملها مع خدمات الحماية الأصلية في نظامي Android و iOS.

ويُسهم هذا النهج الأمني المتقدم في تحجيم مخاطر تسريب البيانات، خصوصاً في سيناريوهات فقدان الجهاز المحمول أو محاولات الاختراق المباشر لملفات التطبيق أو كسر حمايته (Root/Jailbreak) [20].

ويُبين الجدول (5-2) مقارنة تحليلية تبرز الفروق الجوهرية بين أسلوب التخزين التقليدي والتخزين الآمن في بيئة Flutter لتبرير الخيار التقني المعتمد في المشروع.

الجدول 5-2: مقارنة بين Flutter Secure Storage و SharedPreferences

| Flutter Secure Storage (التخزين الآمن) | SharedPreferences (التخزين العادي) | المعيار التقني والأمني           |
|----------------------------------------|------------------------------------|----------------------------------|
| ✓ مدعوم بالكامل                        | X غير مدعوم (نصوص واضحة)           | تشفير البيانات                   |
| موصى به كمعيار أساسي                   | غير موصى به إطلاقاً                | تخزين رموز المصادقة (Tokens)     |
| ✓ يعتمد بشكل أساسي                     | X لا يعتمد                         | الاعتماد على Keystore / Keychain |
| مرتفع (محمي على مستوى النظام)          | متوسط (معرض للاختراق المباشر)      | مستوى الأمان الموفر              |
| ✓ مناسب جداً                           | X غير مناسب                        | ملاءمته للبيانات الحساسة         |

(مقتبس من المرجع [20]).

## 6-2 الخادم الخلفي باستخدام Hono و Cloudflare Workers

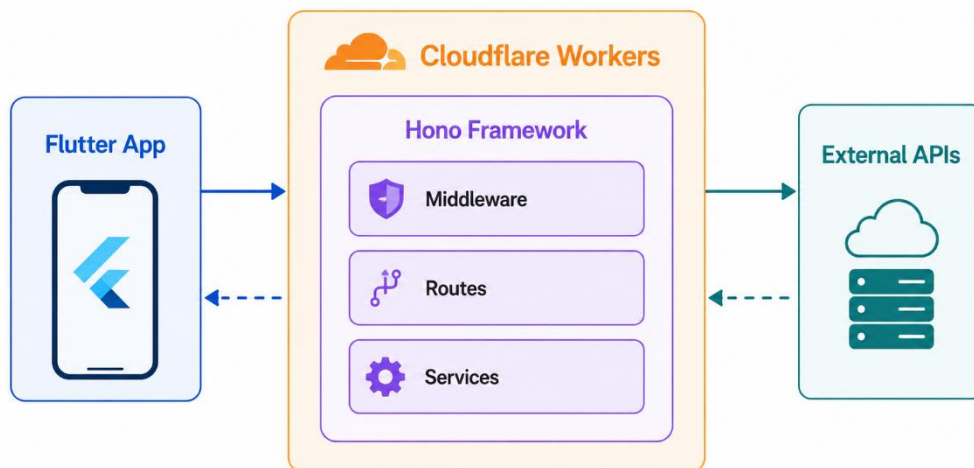
تعتمد هندسة التطبيقات الحديثة على خوادم خلفية (Backend) متينة لتنفيذ منطق الأعمال (Business Logic)، ومعالجة طلبات العملاء، وإدارة التكامل مع قواعد البيانات والخدمات الخارجية. وفي هذا السياق، يبرز Cloudflare Workers كأحد الحلول المتقدمة في مجال الحوسبة عديمة الخوادم (Serverless Computing)؛ إذ تتيح هذه التقنية تنفيذ الشيفرات البرمجية مباشرة على شبكة الحافة (Edge Network) الموزعة عالمياً. يلغي هذا التوجه الحاجة إلى إدارة البنى التحتية للخوادم التقليدية، مما ينعكس إيجاباً على تقليل زمن الاستجابة (Latency) وتحقيق قابلية توسع (Scalability) فورية وعالية الكفاءة [21].

وفي إطار المعمارية الخاصة بمشروع CyberShield، تم الاعتماد على منصة Cloudflare Workers لبناء خادم خلفي يتمتع بالسرعة والموثوقية. يتولى هذا الخادم مهام استقبال طلبات تطبيق Flutter، والتحقق من هويات المستخدمين، وتنفيذ خوارزميات فحص الملفات والروابط، بالإضافة إلى التنسيق مع واجهات برمجة التطبيقات (APIs) الخارجية وإعادة النتائج المهيكلة. ولتنظيم هذا التدفق البرمجي، تم توظيف إطار العمل Hono، وهو إطار متخصص في بناء واجهات (REST APIs) سريعة

وخفيفة. يوفر Hono هيكلية برمجية صارمة لإدارة المسارات (Routes)، ومعالجة الطلبات والاستجابات، وتطبيق الوسطاء (Middleware). ويتجلى ذلك بوضوح في بنية المشروع التي فصلت منطقياً إلى مجلدات مستقلة للمسارات، والخدمات، والوسطاء، إلى جانب ملفات التهيئة الخاصة ببيئة Cloudflare.

ويُكرّس هذا النهج المعماري مبدأ "فصل الاهتمامات" (Separation of Concerns)، حيث يتم عزل منطق الأعمال المعقد وعمليات التحقق المتقدمة داخل الخادم الخلفي، ليقتصر دور تطبيق الهاتف المحمول على تقديم واجهة المستخدم التفاعلية وإرسال الطلبات وعرض النتائج. وهذا العزل الهيكلي يعزز من قابلية صيانة النظام، ويسهل إعادة استخدام الشيفرة البرمجية، ويرفع من مستوى الأمان وكفاءة الأداء العام للمنظومة [22].

يوضح الشكل (6-2) البنية المعمارية لتدفق الطلبات والاستجابات، مبرزاً احتواء شبكة Cloudflare لإطار العمل Hono كبيئة تشغيل مركزية لإدارة العمليات والتكامل مع واجهات البرمجة الخارجية (External APIs).



الشكل 6-2: البنية المعمارية للخادم الخلفي باستخدام إطار عمل Hono و Cloudflare Workers.

ويُفصّل الجدول (6-2) المهام الوظيفية الأساسية التي يضطلع بها كل مكون من مكونات الخادم الخلفي ضمن معمارية النظام.

الجدول 6-2: لتوزيع الوظيفي لمكونات الخادم الخلفي في مشروع CyberShield

| المكون (Component)   | الوظيفة التقنية والتشغيلية (Technical Function)                                           |
|----------------------|-------------------------------------------------------------------------------------------|
| Cloudflare Workers   | توفير بيئة تشغيل لا مركزية على شبكة الحافة (Edge) لمعالجة الطلبات دون خوادم تقليدية.      |
| إطار العمل Hono      | بناء واجهات REST APIs وإدارة وتوجيه المسارات (Routing) بكفاءة عالية.                      |
| الوسطاء (Middleware) | اعتراض الطلبات للتحقق من المصادقة، وإدارة الجلسات، ومعالجة الأخطاء برمجياً.               |
| الخدمات (Services)   | تنفيذ منطق الأعمال الأساسي (Business Logic) والتكامل الآمن مع الخدمات والمكتبات الخارجية. |

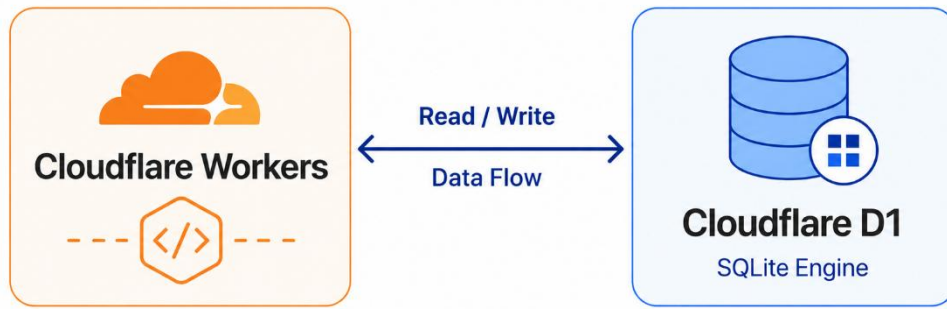
## 7-2 قاعدة البيانات باستخدام Cloudflare D1

تُعدّ قواعد البيانات الركيزة الأساسية لأي نظام برمجي متكامل؛ إذ تتولى مهام التخزين، والاسترجاع، والإدارة المهيكلية للبيانات بما يضمن سلامتها وموثوقية الوصول إليها. وفي ظل التطور المتسارع لهندسة البرمجيات، تتجه التطبيقات الحديثة نحو تبني قواعد بيانات سحابية المنشأ (Cloud-Native) توفر أداءً فائقاً وتكاملاً سلساً مع الخوادم الطرفية، مما يسهم بشكل مباشر في تبسيط دورة حياة التطوير وتقليل زمن الاستجابة (Latency) إلى حدوده الدنيا [23].

وفي إطار المعمارية التقنية لمشروع CyberShield، جرى اعتماد Cloudflare D1 كقاعدة بيانات علائقية رئيسية للنظام. وُظّفت هذه القاعدة مركزياً لإدارة وحفظ هويات المستخدمين، وتوثيق السجلات التاريخية لعمليات الفحص، وتخزين نتائج التحليل الأمني الشامل. وتتميز D1 بتكاملها

المباشر والآتي مع شبكة Cloudflare Workers، حيث تتم معالجة أوامر القراءة والكتابة ( I/O Operations) حصرياً عبر طبقة الخدمات (Services) في الخادم الخلفي، مما ينفي الحاجة إلى تهيئة وصيانة خوادم قواعد بيانات مستقلة. ولضمان استدامة البنية وقابليتها للتطوير، نُظمت قاعدة البيانات برمجياً عبر ملفات مخصصة لإنشاء الجداول وإدارة عمليات الترحيل (Migrations) بشكل آلي ومنهجي.

ويوضح الشكل (7-2) الآلية الهندسية لتكامل Cloudflare D1 مع شبكة Cloudflare Workers لتنفيذ الاستعلامات وإدارة البيانات المركزية.



الشكل 7-2: تكامل الخادم الخلفي Cloudflare Workers مع قاعدة البيانات السحابية Cloudflare D1.

ومن الناحية الهندسية، تعتمد Cloudflare D1 في بنيتها التحتية على محرك SQLite، مما يمنحها خفة ومرونة استثنائية في إدارة البيانات مع الاحتفاظ بالدعم الكامل للغة الاستعلامات البنيوية القياسية (SQL). هذا الدمج بين مرونة SQLite وقوة الحوسبة عديمة الخوادم (Serverless) يجعل من D1 خياراً مثالياً لتلبية متطلبات التطبيقات الموزعة عالية الأداء [24].

ويُبين الجدول (7-2) أبرز الخصائص الفنية والتشغيلية التي تميز قاعدة بيانات Cloudflare D1 لتبرير استخدامها ضمن بنية النظام.

الجدول 2-7: الخصائص الرئيسية لقاعدة البيانات Cloudflare D1

| الخاصية                       | الوصف التقني في إطار Cloudflare D1                |
|-------------------------------|---------------------------------------------------|
| محرك قاعدة البيانات           | SQLite                                            |
| نموذج التشغيل                 | الحوسبة عديمة الخوادم (Serverless) الموزعة طرفياً |
| التكامل مع Cloudflare Workers | تكامل أصيل ومباشر (Native Integration)            |
| لغة الاستعلام                 | مدعومة بالكامل (Standard SQL)                     |
| إدارة الترحيل (Migrations)    | مدعومة عبر أدوات سطر الأوامر (CLI)                |
| قابلية التوسع                 | عالية ومرنة وتلقائية (Auto-scaling)               |

(مقتبس من المرجع [24]).

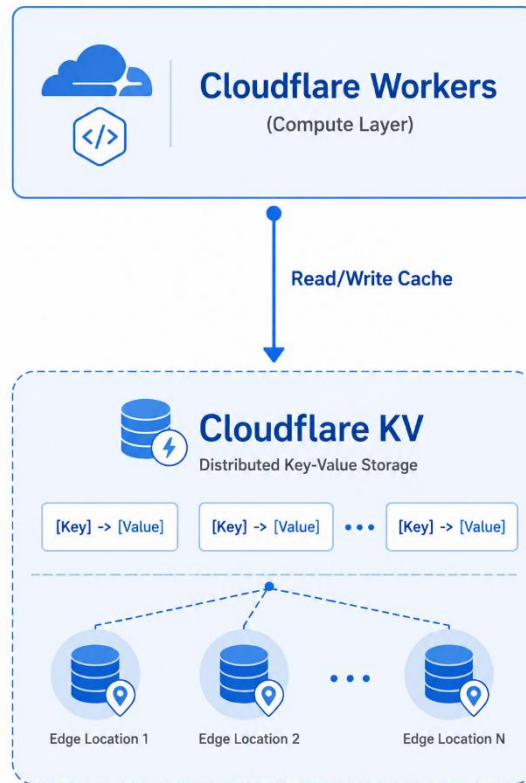
## 8-2 التخزين المؤقت باستخدام Cloudflare KV

يُعدّ التخزين المؤقت (Caching) تقنية محورية في الارتقاء بأداء التطبيقات السحابية؛ إذ يهدف إلى تقليل زمن الاستجابة عبر الاحتفاظ بالبيانات كثيرة الاستخدام في الذاكرة المخبئية، مما يلغي الحاجة إلى تكرار العمليات البرمجية المعقدة عند كل طلب. وفي هذا الإطار، توفر خدمة Cloudflare KV مخزناً بياناتياً موزعاً يعتمد على بنية المفتاح والقيمة (Key-Value)، مما يتيح حفظ البيانات واسترجاعها بسرعات فائقة عبر شبكة Cloudflare العالمية الطرفية (Edge Network)، وهو ما يسهم بشكل مباشر في تخفيف العبء عن الخدمات الخلفية [25].

وفي المعمارية الهندسية لمشروع CyberShield، تم توظيف خدمة Cloudflare KV كطبقة تخزين مؤقت للبيانات التي لا تتطلب تحديثاً متزامناً، بغية تقليل حجم الطلبات الموجهة للخدمات الخارجية ورفع كفاءة الخادم الخلفي. وتتكامل هذه الخدمة بشكل أصيل ومباشر مع بيئة Cloudflare Workers، حيث تتم إدارة البيانات المخزنة والوصول إليها من داخل الخادم الخلفي عبر واجهات برمجية خفيفة، مما يعزز أداء التطبيق عند تنفيذ العمليات الاستعلامية المتكررة ويخفض زمن

الوصول بشكل ملحوظ. وقد تجلّى هذا التكامل بوضوح ضمن بنية إعدادات الخادم الخلفي وملفات التهيئة المعتمدة في المشروع.

ويوضح الشكل (8-2) الآلية الهندسية لهذا التكامل وتدفق البيانات بين Cloudflare Workers وخدمة التخزين الموزع.



الشكل 8-2: تكامل الخادم الخلفي Cloudflare Workers مع خدمة التخزين المؤقت Cloudflare KV.

وتبرز أهمية Cloudflare KV في سلاسة اندماجها مع تطبيقات الحوسبة عديمة الخوادم (Serverless)، إلى جانب كفاءتها العالية في التوزيع الجغرافي للبيانات، مما يجعلها الخيار الأمثل للتطبيقات التي تركز على معدلات قراءة مرتفعة (Read-Heavy) مقارنة بعمليات الكتابة والتحديث [26].

ويُفصل الجدول (8-2) المقارنة الفنية بين خدمة Cloudflare KV وقاعدة البيانات Cloudflare D1، لتوضيح التباين في حالات الاستخدام الوظيفي داخل بنية النظام.

الجدول 8-2: مقارنة بين Cloudflare KV و Cloudflare D1

| المعيار التقني     | Cloudflare KV (التخزين المؤقت)           | Cloudflare D1 (قاعدة البيانات الرئيسية)        |
|--------------------|------------------------------------------|------------------------------------------------|
| نوع التخزين        | المفتاح والقيمة (Key-Value)              | قاعدة بيانات علائقية (Relational Database)     |
| نموذج البيانات     | مفاتيح وقيم بسيطة                        | جداول، حقول، وعلاقات                           |
| لغة الاستعلام      | واجهات برمجية بسيطة (لا تتطلب SQL)       | تعتمد بالكامل على لغة الاستعلام القياسية (SQL) |
| الاستخدام الأمثل   | البيانات كثيفة القراءة (Read-Heavy Data) | البيانات العلائقية والمهيكلية                  |
| التكامل مع Workers | تكامل مباشر وأصيل                        | تكامل مباشر وأصيل                              |

(مقتبس من المرجع [25]).

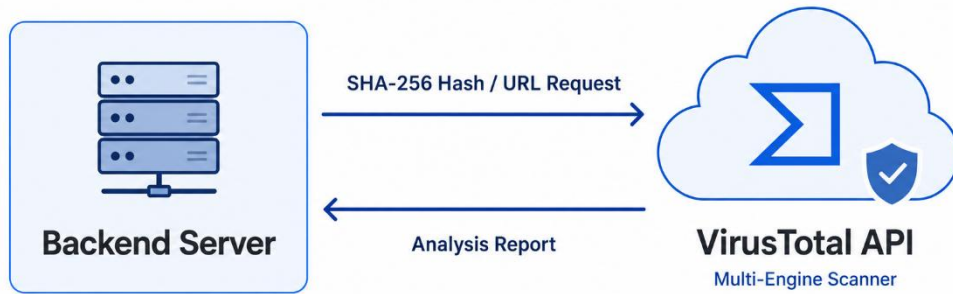
## 9-2 التكامل مع VirusTotal API

يُعدّ VirusTotal من أبرز المنصات السحابية المتخصصة في تحليل الملفات والروابط المشبوهة، حيث يركز عمله على تجميع ومقاطعة نتائج العشرات من محركات مكافحة البرمجيات الخبيثة. توفر هذه الآلية تقييماً أمنياً شاملاً ودقيقاً يفوق ما يقدمه محرك فحص منفرد. وتتيح المنصة واجهات برمجة تطبيقات (APIs) متطورة لتمكين الأنظمة الخارجية من أتمتة عمليات الفحص واسترجاع التقارير الأمنية بكفاءة عالية، مما يجعلها خياراً استراتيجياً للتكامل مع التطبيقات الأمنية الحديثة [27].

وفيما يخص البنية التشغيلية لمشروع CyberShield، جرى توظيف VirusTotal API كعصب رئيسي لتنفيذ عمليات الفحص والتقصي. وتعتمد الآلية على إرسال تطبيق الهاتف للبيانات المراد

فحصها إلى الخادم الخلفي، الذي يتولى بدوره إدارة الاتصال الآمن مع واجهة VirusTotal، ومن ثم معالجة استجابة الخادم السحابي وإعادة توجيه النتائج النهائية المهيكلة إلى واجهة المستخدم. يضمن هذا العزل المعماري الاستفادة القصوى من قدرات المحركات السحابية المتعددة دون إثقال الهاتف المحمول بمهام الفحص المحلي، مما يرفع من دقة التقييم الأمني ويقلل من استهلاك الموارد المادية للجهاز.

ويوضح الشكل (9-2) مسار تدفق البيانات وآلية التكامل الهندسي بين الخادم الخلفي وخدمة VirusTotal API لتنفيذ مهام الفحص.



الشكل 9-2: آلية فحص الملفات والروابط والتكامل مع واجهة VirusTotal API.

ولتعزيز الخصوصية وتقليل استهلاك النطاق الترددي (Bandwidth) أثناء فحص الملفات، يعتمد النظام على خطوة استباقية تتمثل في توليد بصمة رقمية فريدة (SHA-256 Hash) للملف. وتُرسَل هذه البصمة كمعيار أولي للاستعلام عن حالة الملف ضمن قاعدة بيانات VirusTotal؛ فإذا كانت البصمة معروفة مسبقاً، يُسترجع التقرير الأمني مباشرة دون الحاجة لرفع الملف الفعلي. أما في حال فحص الروابط (URLs)، فتُمرر مباشرة إلى الخدمة لإجراء تحليل ديناميكي واستخراج مؤشرات الخطورة (IoCs) المرتبطة بها [28].

ويُبيّن الجدول (9-2) التصنيفات الوظيفية لأهم عمليات الفحص التي توفرها واجهة VirusTotal API وتم توظيفها في النظام.

الجدول 2-9: أبرز خدمات VirusTotal API

| الخدمة (Service)                     | الاستخدام والتوظيف التقني                                                     |
|--------------------------------------|-------------------------------------------------------------------------------|
| تحليل الملفات (File Analysis)        | رفع وتحليل الملفات غير المعروفة عبر محركات الفحص المتعددة.                    |
| تحليل الروابط (URL Analysis)         | فحص الروابط للتحقق من خلوها من برمجيات التصيد أو البرمجيات الخبيثة.           |
| الاستعلام بالبصمة (File Hash Lookup) | استرجاع تقارير الملفات المعروفة باستخدام خوارزميات التشفير (SHA-256, MD5).    |
| تقارير التحليل (Analysis Reports)    | استرجاع النتائج التفصيلية ومعدل الخطورة (Rate Detection) من المحركات الأمنية. |

(مقتبس من المرجع [27]).

## 10-2 التكامل مع Google Gemini API

يُصنّف Google Gemini كأحد أبرز نماذج الذكاء الاصطناعي التوليدي متعدد الوسائط (Multimodal Generative AI)، حيث يتميز بقدرات فائقة على فهم السياقات وتحليل البيانات المعقدة وإنتاج استجابات طبيعية بالاعتماد على معمارية نماذج اللغة الكبيرة (LLMs). ولتوسيع نطاق الاستفادة من هذه القدرات، توفر المنصة واجهة برمجة تطبيقات (Gemini API) تتيح للمطورين دمج محرك الذكاء الاصطناعي برمجياً داخل أنظمتهم، مما يفتح المجال لأتمتة عمليات التحليل وتوليد المحتوى التفاعلي بكفاءة عالية [29].

وفي سياق مشروع CyberShield، تم توظيف واجهة Google Gemini API كطبقة تحليل ذكية لمعالجة النتائج الفنية الخام الواردة من خدمات الفحص (مثل VirusTotal). فعوضاً عن إرباك المستخدم النهائي ببيانات تقنية معقدة، تُمرر هذه النتائج إلى نموذج Gemini، الذي يتولى تحليلها وإعادة صياغتها برمجياً لإنتاج تقرير أمني مبسط. يتضمن هذا التقرير تقييماً واضحاً لمستوى الخطورة، وتفسيراً منطقياً لأسباب التصنيف، بالإضافة إلى حزمة من التوصيات الأمنية القابلة للتنفيذ. ويتجلى

هذا الدمج هيكلياً في وجود خدمة مستقلة (Service) داخل الخادم الخلفي، تُعنى بإدارة الاتصال وتأمين الطلبات وتميرير المعطيات المحددة ضمن ملفات التهيئة (Configuration Files).

ومن الناحية البرمجية، تركز دورة التكامل هذه على تجميع البيانات الفنية وتغليفها ضمن حزمة بصيغة (JSON) وإرسالها عبر قناة آمنة إلى واجهة Gemini API. وبمجرد معالجة النموذج للبيانات، يُسترد التقرير النصي ليقوم الخادم الخلفي بتوجيهه فوراً إلى تطبيق الهاتف لعرضه ضمن واجهة المستخدم. وتُسهّم هذه المعمارية المتقدمة في الارتقاء بتجربة المستخدم (UX)، حيث تُترجم المؤشرات الأمنية المعقدة إلى معلومات استرشادية واضحة [30].

ويُبيّن الجدول (2-10) التسلسل المنطقي لمراحل معالجة البيانات وتدفعها عند توظيف واجهة Gemini API داخل بنية النظام.

الجدول 2-10: مراحل عمل Google Gemini API في مشروع CyberShield

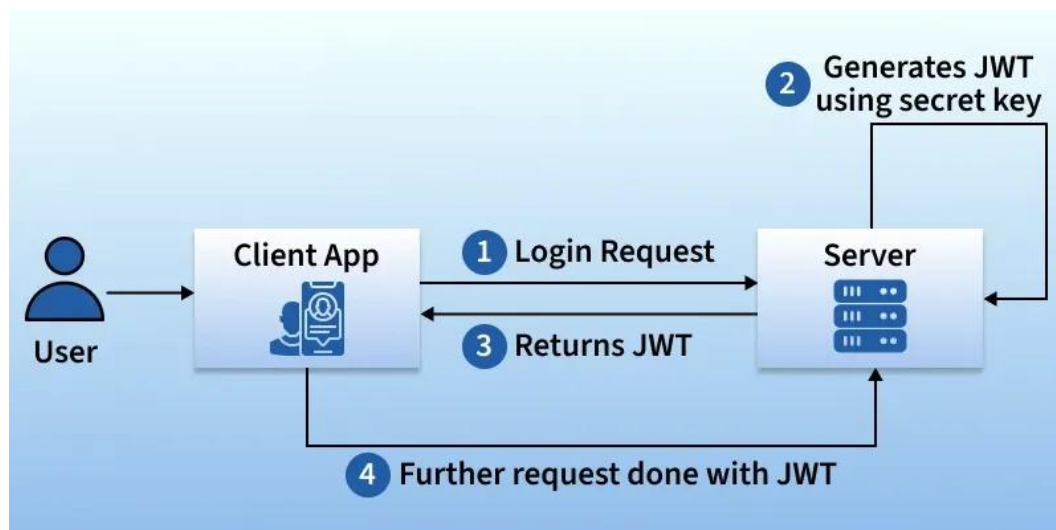
| المرحلة (Phase)                      | التوصيف التقني والوظيفي                                                     |
|--------------------------------------|-----------------------------------------------------------------------------|
| إرسال البيانات (Data Transmission)   | تغليف النتائج الفنية للفحص وإرسالها إلى واجهة Gemini API بصيغة JSON.        |
| المعالجة والتحليل (Model Processing) | استيعاب البيانات الواردة وتحليلها سياقياً بواسطة نموذج الذكاء الاصطناعي.    |
| توليد التقرير (Report Generation)    | إنتاج نص تحليلي يحدد مستوى الخطورة ويقدم توصيات أمنية مخصصة.                |
| إعادة الاستجابة (Response Delivery)  | تصدير التقرير النهائي إلى الخادم الخلفي ليمرر بدوره إلى واجهة تطبيق الهاتف. |

## 11-2 المصادقة باستخدام JSON Web Token

تُعدّ آلية المصادقة (Authentication) الجدار الأمني الأول والركيزة الأساسية في تطبيقات الهواتف المحمولة، إذ تضمن التحقق الموثوق من هوية المستخدمين قبل منحهم صلاحيات الوصول إلى الموارد والخدمات المحمية. وفي هذا السياق، يبرز معيار JSON Web Token (JWT) كأحد أكثر الحلول كفاءة واعتمادية في أنظمة المصادقة الحديثة؛ حيث يعتمد على توليد رمز مميز (Token) مدمج يحتوي على حزمة من البيانات والصلاحيات (Claims) الموقعة تشفيرياً. تتيح هذه الآلية للخوادم التحقق من مصداقية وهوية المستخدم بشكل مستقل، دون الحاجة إلى تخزين أو تتبع حالة الجلسات (Sessions) في قاعدة البيانات [31].

وفي البنية المعمارية لمشروع CyberShield، تم اعتماد معيار JWT كآلية مركزية لإدارة جلسات المصادقة بين تطبيق الهاتف المحمول والخادم الخلفي. تبدأ دورة حياة المصادقة بتوليد رمز مميز فور نجاح عملية تسجيل الدخول، يُرسل إلى التطبيق حيث يُحفظ في بيئة معزولة ومحماة باستخدام مكتبة (Flutter Secure Storage). لاحقاً، يُوظف هذا الرمز كوثيقة عبور رقمية ترافق كل طلب استعلامي يُوجه للخادم؛ إذ يُدمج ضمن ترويسة الطلبات (Authorization Header) للتحقق من امتلاك المستخدم للصلاحيات اللازمة لتنفيذ المهام الحساسة، كإجراء عمليات الفحص، أو استعراض السجلات التاريخية، أو تعديل بيانات الحساب. ولضمان تطبيق هذه المعايير بصرامة، زُوّد الخادم الخلفي ببرمجيات وسيطة (Middleware) تتولى مهمة اعتراض الطلبات الواردة وفك تشفير الرمز للتحقق من سلامته وصلاحيته قبل تمرير الطلب إلى طبقة الخدمات (Services).

ويوضح الشكل (2-10) دورة حياة المصادقة وتدفق البيانات بين تطبيق الهاتف المحمول والخادم الخلفي باستخدام آلية JSON Web Token.



الشكل 2-10: دورة المصادقة وتدفق الرموز المميزة (JWT) بين تطبيق العميل والخادم.

وتكمن القوة الهندسية لمعيار JWT في خفة حجمه وسهولة تمريره عبر بروتوكول HTTP، فضلاً عن دعمه الكامل لبناء أنظمة مصادقة عديمة الحالة (Stateless Authentication). يُسهم هذا النهج بشكل مباشر في تقليص العبء الحسابي على الخوادم ورفع قابلية التوسع (Scalability) للنظام، متفوقاً بذلك على آليات إدارة الجلسات التقليدية [32].

ويُفصّل الجدول (2-11) المكونات البنوية الثلاثة التي يتألف منها الرمز المميز (JWT)، والتي تضمن موثوقيته وسلامته من التلاعب.

الجدول 2-11: المكونات البنوية لرمز JSON Web Token (JWT)

| المكون (Component)  | التوصيف الفني والوظيفي                                                                                              |
|---------------------|---------------------------------------------------------------------------------------------------------------------|
| الترويسة (Header)   | تحدد نوع الرمز (JWT) وخوارزمية التشفير المستخدمة في التوقيع (مثل HMAC SHA256).                                      |
| الحمولة (Payload)   | تحتوي على المطالبات (Claims) التي تمثل بيانات المستخدم الجوهرية وصلاحياته وتاريخ انتهاء الرمز.                      |
| التوقيع (Signature) | يُولّد تشفيرياً لضمان سلامة البيانات (Data Integrity) والتحقق من عدم تعرض الرمز لأي تلاعب أو تعديل من أطراف خارجية. |

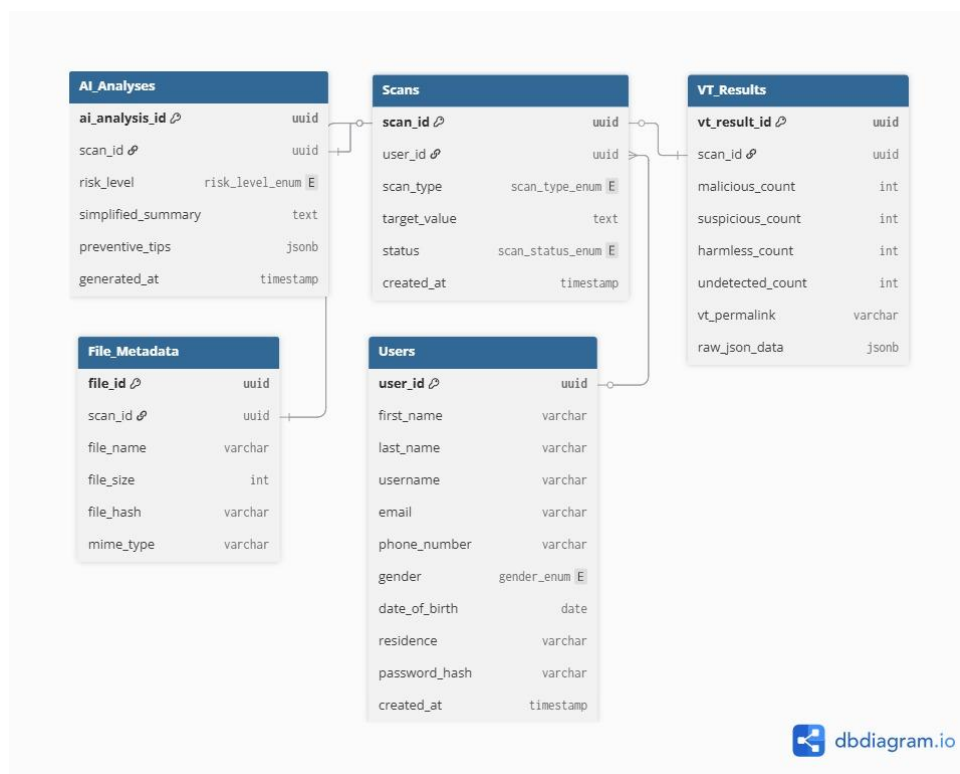
الفصل الثالث: تحليل

وتصميم النظام

### 1-3 تحليل قاعدة البيانات

تمثل قاعدة البيانات العنصر الرئيس المسؤول عن تنظيم البيانات وإدارتها داخل نظام CyberShield، حيث تعتمد على نموذج الكيانات والعلاقات (Entity Relationship Diagram - ERD) لتوضيح الجداول الرئيسة والعلاقات فيما بينها. ويهدف هذا النموذج إلى ضمان تنظيم البيانات وتقليل التكرار والمحافظة على سلامة البيانات أثناء عمليات التخزين والاسترجاع، بالإضافة إلى دعم جميع العمليات التي ينفذها النظام، مثل إدارة المستخدمين، وتسجيل عمليات الفحص، وحفظ نتائج التحليل الأمني والتقارير المولدة.

يوضح الشكل (1-3) مخطط الكيانات والعلاقات (ERD) الخاص بقاعدة بيانات النظام، والذي يتكون من خمس جداول مترابطة هي: Users و Scans و VT\_Results و AI\_Analyses و File\_Metadata. حيث يمثل كل جدول جزءًا محددًا من البيانات اللازمة لتنفيذ وظائف النظام.



الشكل 1-3: مخطط الكيانات والعلاقات (Entity Relationship Diagram) لقاعدة بيانات نظام CyberShield.

### 3-1-1 جدول المستخدمين (Users)

يمثل جدول Users نقطة البداية في قاعدة البيانات، إذ يحتفظ بالبيانات الأساسية الخاصة بالمستخدمين المسجلين في النظام، مثل الاسم، واسم المستخدم، والبريد الإلكتروني، ورقم الهاتف، والجنس، وتاريخ الميلاد، ومكان الإقامة، بالإضافة إلى كلمة المرور بعد تشفيرها. ويستخدم الحقل user\_id كمفتاح أساسي (Primary Key) لتمييز كل مستخدم بصورة فريدة، كما يرتبط هذا الجدول بجدول Scans من خلال علاقة واحد إلى متعدد (One-to-Many)، حيث يستطيع المستخدم تنفيذ عدد غير محدود من عمليات الفحص.

### 3-1-2 جدول عمليات الفحص (Scans)

يمثل جدول Scans المحور الرئيس الذي ترتبط به بقية جداول قاعدة البيانات، إذ يسجل جميع عمليات الفحص التي يجريها المستخدم، سواء كانت فحص ملفات أو روابط إلكترونية. ويتضمن الجدول معرف العملية (scan\_id) كمفتاح أساسي، إضافة إلى معرف المستخدم (user\_id) كمفتاح أجنبي يشير إلى صاحب عملية الفحص، إلى جانب نوع الفحص، والقيمة المستهدفة، وحالة العملية، وتاريخ إنشائها.

ويرتبط هذا الجدول مباشرةً بجدول VT\_Results و AI\_Analyses و File\_Metadata، مما يجعله نقطة الربط الرئيسة بين بيانات الفحص ونتائج التحليل المختلفة.

### 3-1-3 جدول نتائج VirusTotal (VT\_Results)

يُستخدم جدول VT\_Results لتخزين النتائج المسترجعة من خدمة VirusTotal API بعد انتهاء عملية الفحص. ويتضمن الجدول معلومات إحصائية عن عدد المحركات التي صنفت الملف أو الرابط

على أنه خبيث، أو مشبوه، أو غير ضار، أو غير مكتشف، بالإضافة إلى رابط التقرير الأصلي (vt\_permalink) والبيانات الخام المستلمة بصيغة JSON.

ويرتبط هذا الجدول بجدول Scans بواسطة المفتاح الأجنبي scan\_id، بحيث ترتبط كل نتيجة فحص بعملية الفحص التي أنشأتها.

### 14-3 جدول تحليلات الذكاء الاصطناعي (AI\_Analyses)

يحتفظ جدول AI\_Analyses بالتقارير الأمنية التي يتم توليدها بواسطة نموذج الذكاء الاصطناعي بعد تحليل نتائج VirusTotal. ويتضمن مستوى الخطورة، والملخص المبسط، والإرشادات الوقائية، بالإضافة إلى تاريخ إنشاء التقرير.

ويرتبط هذا الجدول بجدول Scans من خلال الحقل scan\_id، بحيث يمتلك كل تقرير تحليلي علاقة مباشرة بعملية الفحص التي استند إليها.

### 5-1-3 جدول بيانات الملفات (File\_Metadata)

يُستخدم جدول File\_Metadata لتخزين البيانات الوصفية الخاصة بالملفات التي يجري فحصها، مثل اسم الملف، وحجمه، ونوعه، والبصمة الرقمية (Hash). ولا يتضمن هذا الجدول الملف نفسه، وإنما يحتفظ بالبيانات اللازمة لتعريفه وربطه بعملية الفحص.

ويرتبط الجدول بجدول Scans باستخدام الحقل scan\_id، بحيث تُحفظ البيانات الوصفية لكل ملف ضمن عملية الفحص المرتبطة به.

### 6-1-3 العلاقات بين الكيانات

يبين مخطط قاعدة البيانات مجموعة من العلاقات التي تضمن ترابط البيانات بين الجداول المختلفة، ويمكن تلخيصها كما يأتي:

يرتبط جدول Users بجدول Scans بعلاقة واحد إلى متعدد (One-to-Many)، إذ يمكن للمستخدم تنفيذ عدة عمليات فحص، بينما تعود كل عملية فحص إلى مستخدم واحد فقط.

يرتبط جدول Scans بجدول VT\_Results من خلال المفتاح scan\_id، بحيث تُخزن نتيجة VirusTotal الخاصة بكل عملية فحص.

يرتبط جدول Scans بجدول AI\_Analyses، حيث يُنشأ تقرير ذكاء اصطناعي يعتمد على نتائج عملية الفحص.

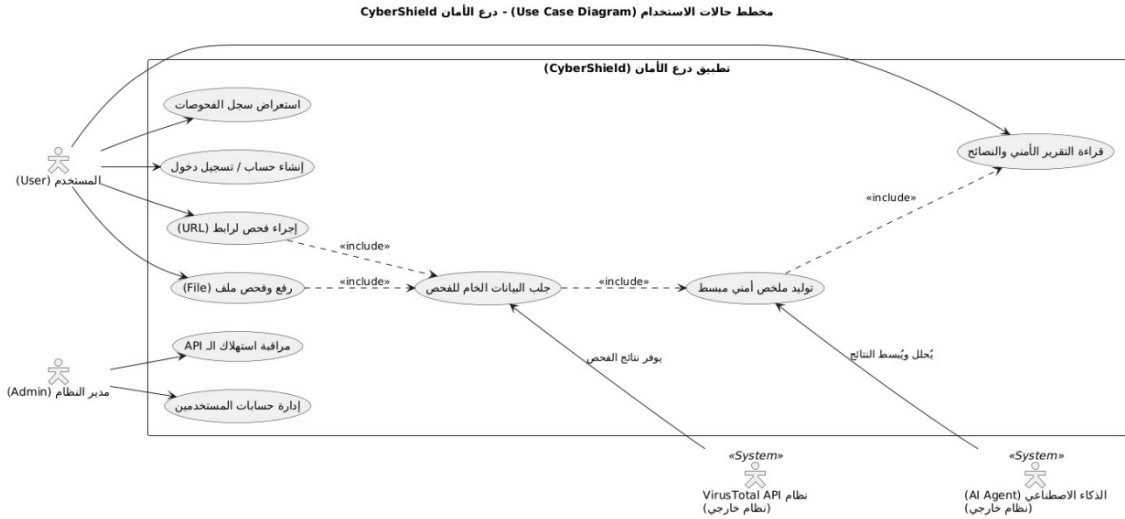
يرتبط جدول Scans بجدول File\_Metadata لحفظ البيانات الوصفية الخاصة بالملفات التي تم فحصها.

وتسهم هذه العلاقات في المحافظة على تكامل البيانات (Data Integrity)، وتقليل التكرار، وتسهيل الوصول إلى جميع المعلومات المرتبطة بعملية الفحص ابتداءً من المستخدم، مروراً بنتائج VirusTotal، وانتهاءً بالتقرير الأمني الناتج عن الذكاء الاصطناعي.

## 2-3 تحليل حالات الاستخدام

يُستخدم مخطط حالات الاستخدام (Use Case Diagram) لتوضيح التفاعلات الرئيسة بين مستخدم النظام والوظائف التي يوفرها تطبيق CyberShield، حيث يبين الخدمات التي يمكن لكل مستخدم الوصول إليها، إضافة إلى الأنظمة الخارجية التي يتكامل معها النظام لتنفيذ عمليات الفحص وتحليل النتائج.

يوضح الشكل (2-3) مخطط حالات الاستخدام الخاص بالنظام، والذي يضم ممثلين رئيسيين هما المستخدم (User) ومدير النظام (Admin)، بالإضافة إلى نظامي VirusTotal API ووكيل الذكاء الاصطناعي (AI Agent) بوصفهما نظامين خارجيين يقدمان خدمات الفحص والتحليل.



الشكل 3-2: مخطط حالات الاستخدام (Use Case Diagram) لنظام CyberShield.

### 1-2-3 المستخدم (User)

يمثل المستخدم الطرف الرئيس الذي يتعامل مع التطبيق، إذ يستطيع الاستفادة من الخدمات الأساسية التي يوفرها النظام بعد إنشاء حساب وتسجيل الدخول. وتشمل هذه الخدمات تنفيذ عمليات فحص الملفات والروابط، والاطلاع على نتائج الفحص، إضافة إلى مراجعة سجل العمليات السابقة.

وتبدأ عملية الاستخدام بإنشاء حساب جديد أو تسجيل الدخول إلى النظام، ثم يختار المستخدم نوع الفحص المطلوب، سواء كان فحص ملف أو فحص رابط إلكتروني. بعد انتهاء عملية الفحص، يعرض النظام تقريراً أمنياً مبسطاً يوضح نتيجة التحليل، مع إمكانية الرجوع إلى سجل الفحوصات السابقة في أي وقت.

### 2-2-3 مدير النظام (Administrator)

يمثل مدير النظام المسؤول عن متابعة الجوانب الإدارية للتطبيق، حيث يتيح له النظام إدارة حسابات المستخدمين ومراقبة استهلاك واجهات برمجة التطبيقات (API Usage)، بما يضمن استمرار عمل النظام بكفاءة وإدارة الموارد المتاحة بصورة مناسبة.

ولا يتداخل مدير النظام مع عمليات الفحص نفسها، وإنما يقتصر دوره على إدارة النظام ومتابعة تشغيل الخدمات.

### 3-2-3 الأنظمة الخارجية (External Systems)

يتكامل النظام مع نظامين خارجيين لتنفيذ الوظائف التي تتطلب خدمات خارجية.

النظام الأول هو VirusTotal API، والذي يستقبل بيانات الفحص من الخادم الخلفي ويعيد النتائج الأمنية الخاصة بالملفات أو الروابط بعد تحليلها بواسطة محركات الفحص المختلفة. وتمثل هذه النتائج المصدر الأساسي للمعلومات الأمنية التي يعتمد عليها النظام.

أما النظام الثاني فهو وكيل الذكاء الاصطناعي (AI Agent)، والذي يستقبل نتائج الفحص الأولية ويقوم بتحليلها لإنتاج تقرير مبسط يتضمن مستوى الخطورة وملخصًا للنتائج، إضافة إلى مجموعة من الإرشادات الوقائية التي تساعد المستخدم على فهم التقرير واتخاذ القرار المناسب.

### 4-2-3 العلاقات بين حالات الاستخدام

يبين المخطط مجموعة من العلاقات بين الوظائف المختلفة داخل النظام، حيث تعتمد عمليتا رفع وفحص الملف وإجراء فحص الرابط على وظيفة جلب البيانات الخام للفحص باستخدام علاقة <<include>>، لأن كلتا العمليتين تحتاجان إلى الحصول على نتائج الفحص قبل متابعة بقية خطوات المعالجة

كما تعتمد وظيفة توليد ملخص أمني مبسط على البيانات الخام المستلمة من خدمة VirusTotal API، إذ تُرسل هذه النتائج إلى وكيل الذكاء الاصطناعي لتحليلها وإنتاج تقرير مبسط يمكن للمستخدم فهمه بسهولة.

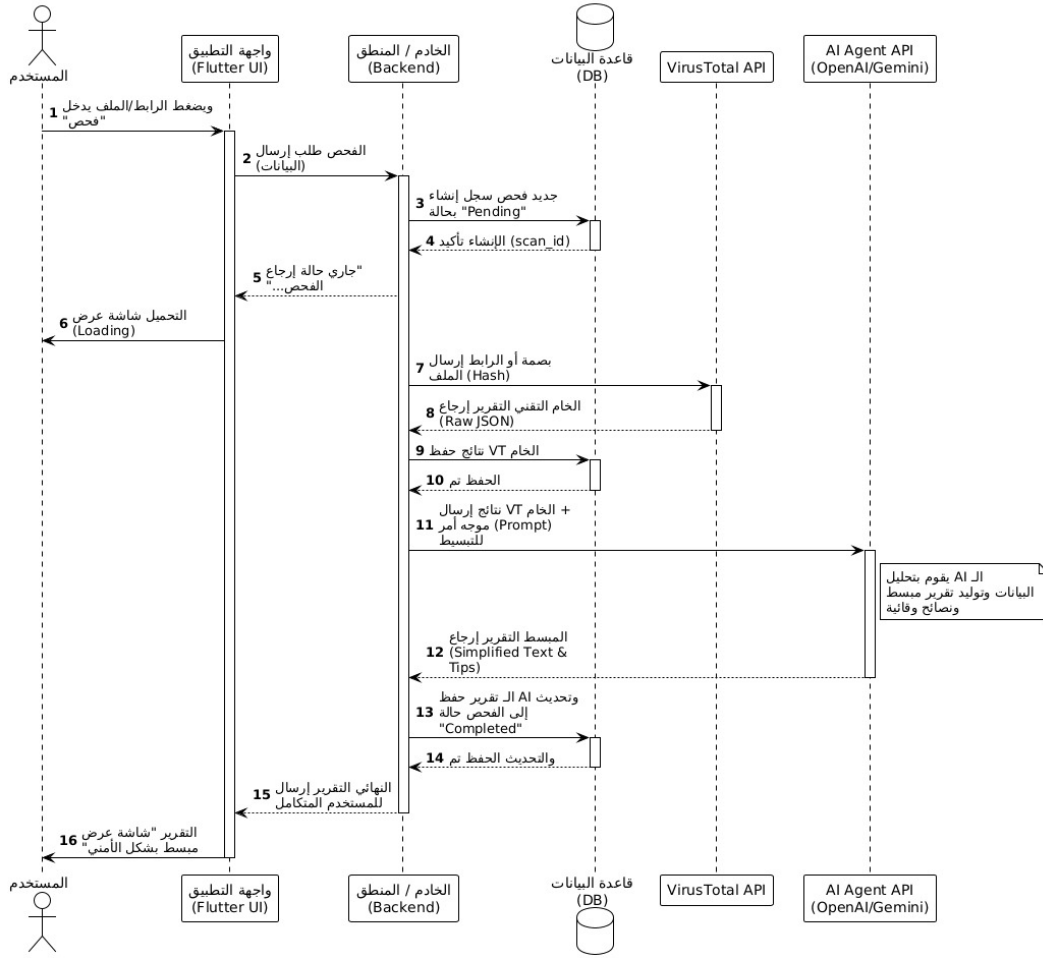
بعد ذلك ترتبط وظيفة قراءة التقرير الأمني والنصائح مباشرةً بعملية توليد الملخص الأمني، بحيث يتمكن المستخدم من الاطلاع على التقرير النهائي بعد انتهاء جميع مراحل الفحص والتحليل. وتوضح هذه العلاقات أن تنفيذ عمليات الفحص يعتمد على تسلسل منطقي يبدأ بالحصول على نتائج الفحص، ثم تحليلها، وأخيرًا عرض التقرير النهائي للمستخدم.

### 3-3 تحليل مخطط التسلسل

يُستخدم مخطط التسلسل (Sequence Diagram) لتوضيح التتابع الزمني للرسائل المتبادلة بين مكونات النظام أثناء تنفيذ عملية الفحص، ابتداءً من إدخال المستخدم للملف أو الرابط، وانتهاءً بعرض التقرير الأمني النهائي. ويبين المخطط كيفية انتقال البيانات بين واجهة التطبيق، والخادم الخلفي، وقاعدة البيانات، وخدمة VirusTotal API، إضافة إلى وكيل الذكاء الاصطناعي المسؤول عن توليد التقرير المبسط.

يوضح الشكل (3-3) التسلسل الكامل لعملية الفحص وتحليل النتائج داخل نظام CyberShield.

مخطط التسلسل (Sequence Diagram) - عملية الفحص والتحليل



الشكل 3-3: مخطط التسلسل (Sequence Diagram) لعملية الفحص والتحليل في نظام CyberShield.

### 1-3-3 بدء عملية الفحص

تبدأ العملية عندما يقوم المستخدم بإدخال ملف أو رابط إلكتروني داخل التطبيق، ثم يضغط على زر الفحص. تستقبل واجهة التطبيق هذا الطلب، وتقوم بإرسال البيانات إلى الخادم الخلفي لمعالجة عملية الفحص.

وعند استلام الطلب، ينشئ الخادم الخلفي سجلاً جديداً داخل قاعدة البيانات يمثل عملية الفحص الحالية، وتُحدد حالته الأولية على أنها Pending، ثم يعيد معرف العملية (scan\_id) إلى التطبيق ليتم استخدامه خلال بقية مراحل التنفيذ.

وبعد ذلك يعرض التطبيق شاشة تحميل لإعلام المستخدم بأن عملية الفحص ما تزال قيد التنفيذ.

### 2-3-3 تنفيذ عملية الفحص

بعد إنشاء سجل الفحص، يبدأ الخادم الخلفي بالتواصل مع خدمة VirusTotal API، حيث يرسل بصمة الملف (Hash) أو الرابط المطلوب فحصه وفقًا لنوع العملية. وبعد انتهاء الخدمة من تنفيذ الفحص، تعيد التقرير الخام بصيغة JSON إلى الخادم الخلفي.

ويقوم الخادم بحفظ البيانات المستلمة داخل قاعدة البيانات وربطها بسجل الفحص الذي أنشئ في بداية العملية، وذلك لضمان الاحتفاظ بنتائج الفحص وإمكانية الرجوع إليها لاحقًا.

### 3-3-3 تحليل النتائج بواسطة الذكاء الاصطناعي

بعد تخزين نتائج VirusTotal، يرسل الخادم الخلفي التقرير الخام إلى وكيل الذكاء الاصطناعي مرفقًا بالتعليمات (Prompt) اللازمة لإنتاج تقرير مبسط يمكن للمستخدم فهمه بسهولة.

يقوم وكيل الذكاء الاصطناعي بتحليل النتائج الأمنية، ثم يُولد ملخصًا يتضمن مستوى الخطورة، ووصفًا مختصرًا للحالة، إضافةً إلى مجموعة من النصائح الوقائية المناسبة.

وعند انتهاء عملية التحليل، يعيد وكيل الذكاء الاصطناعي التقرير إلى الخادم الخلفي، والذي يقوم بدوره بحفظه في قاعدة البيانات وتحديث حالة عملية الفحص إلى Completed.

### 4-3-3 عرض التقرير للمستخدم

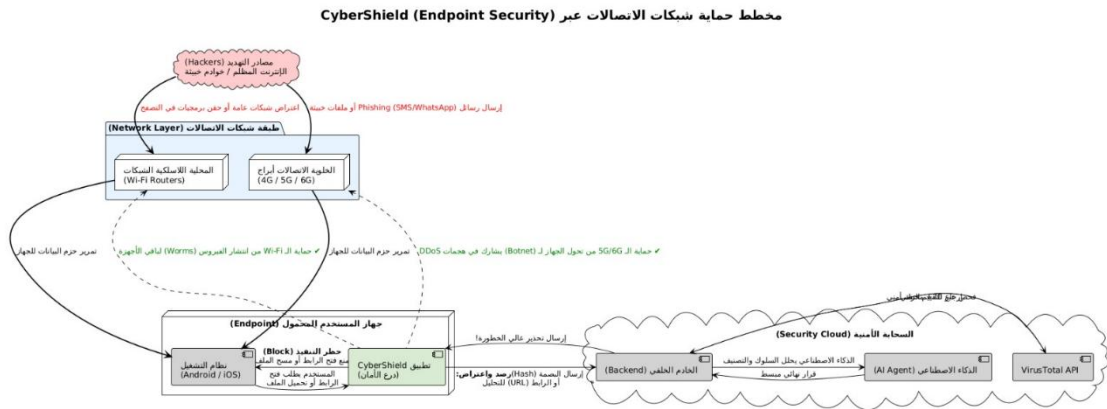
بعد اكتمال جميع مراحل الفحص والتحليل، يرسل الخادم الخلفي التقرير النهائي إلى التطبيق، والذي يعرض للمستخدم ملخصًا آمنًا يتضمن نتيجة الفحص، ومستوى الخطورة، والتوصيات الوقائية المستخلصة من تحليل الذكاء الاصطناعي.

وبذلك تنتهي عملية الفحص بعد انتقال البيانات بين جميع مكونات النظام بصورة متسلسلة، ابتداءً من طلب المستخدم وانتهاءً بعرض التقرير النهائي.

### 4-3 معمارية النظام

تعرض معمارية النظام البنية العامة لنظام CyberShield، وتوضح المكونات الرئيسية للنظام وآلية تفاعلها لتنفيذ عمليات الفحص وتحليل النتائج وعرض التقارير الأمنية للمستخدم. وقد صُممت المعمارية وفق نموذج متعدد الطبقات (Multi-Layer Architecture)، بحيث تُفصل واجهة المستخدم عن منطق الأعمال والخدمات الخارجية، مما يساهم في تحسين تنظيم النظام، وتسهيل عمليات التطوير والصيانة، وزيادة قابلية التوسع.

يوضح الشكل (4-3) المعمارية العامة للنظام، والتي تتكون من أربع طبقات رئيسية هي: طبقة المستخدم، وطبقة التطبيق، وطبقة الخادم الخلفي، وطبقة الخدمات الخارجية.



الشكل 4-3: المعمارية العامة لنظام CyberShield.

### 1-4-3 طبقة المستخدم (User Layer)

تمثل هذه الطبقة نقطة التفاعل المباشر مع النظام، حيث يستخدم المستخدم تطبيق الهاتف المحمول لإجراء عمليات تسجيل الدخول، وفحص الملفات أو الروابط، والاطلاع على نتائج الفحص

والتقارير الأمنية. ولا تحتوي هذه الطبقة على أي منطق لمعالجة البيانات، وإنما يقتصر دورها على استقبال مدخلات المستخدم وعرض النتائج المستلمة من الخادم الخلفي.

### 2-4-3 طبقة التطبيق (Application Layer)

تمثل هذه الطبقة تطبيق الهاتف المحمول المطور باستخدام Flutter، وهي المسؤولة عن إدارة واجهة المستخدم وتنظيم عملية إرسال الطلبات إلى الخادم الخلفي واستقبال الاستجابات منه. كما تتولى إدارة بيانات المستخدم أثناء تشغيل التطبيق، وعرض نتائج الفحص والتقارير الأمنية بصورة مناسبة. وتعمل هذه الطبقة بوصفها وسيطًا بين المستخدم والخادم الخلفي، حيث تجمع البيانات المطلوبة، ثم ترسلها إلى الخادم لمعالجتها قبل عرض النتائج النهائية.

### 3-4-3 طبقة الخادم الخلفي (Backend Layer)

تعد هذه الطبقة المركز الرئيس لمعالجة البيانات داخل النظام، إذ تستقبل جميع الطلبات الواردة من التطبيق، وتنفذ عمليات التحقق من المستخدم، وإدارة بيانات الفحص، والتعامل مع قاعدة البيانات، بالإضافة إلى التواصل مع الخدمات الأمنية الخارجية.

وتتولى هذه الطبقة أيضًا حفظ نتائج الفحص داخل قاعدة البيانات، وإرسال نتائج VirusTotal إلى خدمة الذكاء الاصطناعي لتوليد التقرير الأمني، ثم إعادة التقرير النهائي إلى التطبيق.

### 4-4-3 طبقة الخدمات الخارجية (External Services Layer)

تتكون هذه الطبقة من الخدمات الخارجية التي يعتمد عليها النظام لتنفيذ بعض الوظائف المتخصصة، وتشمل:

VirusTotal API لإجراء عمليات فحص الملفات والروابط واسترجاع النتائج الأمنية. 🛡️

Google Gemini API لتحليل نتائج الفحص وتوليد تقرير أمني مبسط يساعد المستخدم 🤖

على فهم مستوى الخطورة والتوصيات المناسبة.

ويتم التواصل مع هذه الخدمات من خلال الخادم الخلفي فقط، دون اتصال مباشر بينها وبين تطبيق الهاتف المحمول، مما يساهم في حماية مفاتيح الوصول (API Keys) وتنظيم عملية تبادل البيانات.

### 3-4-5 تدفق البيانات داخل النظام

يبدأ تدفق البيانات عندما يرسل المستخدم طلب فحص من خلال تطبيق الهاتف المحمول، فيقوم التطبيق بإرسال الطلب إلى الخادم الخلفي لمعالجته. بعد ذلك يتواصل الخادم مع خدمة VirusTotal API للحصول على نتائج الفحص، ثم يحفظ هذه النتائج في قاعدة البيانات ويرسلها إلى Google Gemini API لتحليلها وإنتاج تقرير أمني مبسط.

وعند اكتمال جميع مراحل المعالجة، يعيد الخادم الخلفي التقرير النهائي إلى التطبيق ليُعرض للمستخدم، مع الاحتفاظ بنتائج الفحص داخل قاعدة البيانات لإتاحتها لاحقاً ضمن سجل الفحوصات. ويحقق هذا التسلسل فصلاً واضحاً بين مسؤوليات مكونات النظام المختلفة، حيث يقتصر دور التطبيق على عرض البيانات والتفاعل مع المستخدم، بينما تُنفذ جميع عمليات المعالجة والتخزين والتكامل مع الخدمات الخارجية داخل الخادم الخلفي.

## الفصل الرابع: التنفيذ العملي

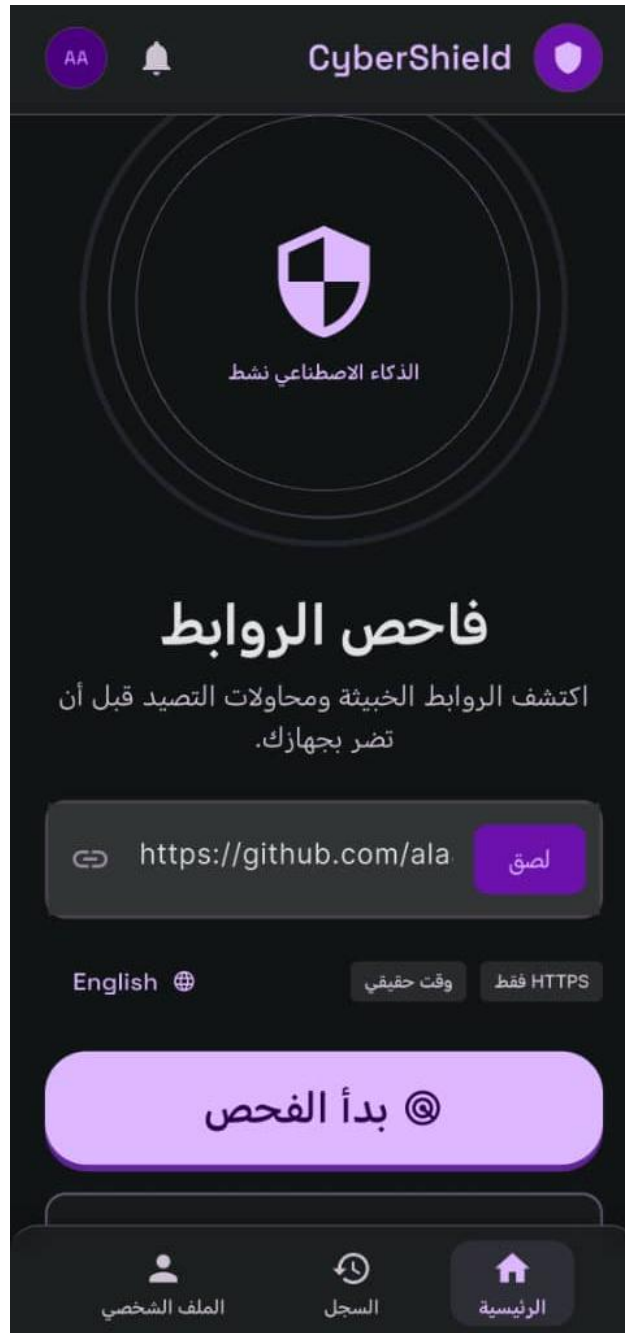
## 1-4 الواجهة الرئيسية للنظام



الشكل 1-4: الواجهة الرئيسية لتطبيق CyberShield.

تمثل هذه الشاشة نقطة البداية بعد تسجيل دخول المستخدم إلى التطبيق، حيث تعرض الحالة العامة للجهاز، بالإضافة إلى الوظائف الرئيسية التي يوفرها النظام. وتتضمن الواجهة خيار فحص الملفات وخيار فحص الروابط، كما تعرض آخر عمليات الفحص المنفذة لتسهيل الوصول إلى النتائج السابقة، مما يوفر للمستخدم نقطة انطلاق لجميع الخدمات الأساسية.

## 2-4 واجهة فحص الروابط



الشكل 2-4: واجهة إدخال الرابط وإعدادات الفحص.

تتيح هذه الشاشة للمستخدم إدخال الرابط الإلكتروني المراد فحصه قبل بدء عملية التحليل، كما توفر بعض الخيارات الإضافية مثل تفعيل فحص روابط HTTPS فقط، واختيار اللغة المناسبة، وإرسال طلب الفحص إلى الخادم الخلفي عند الضغط على زر بدء الفحص.

## 3-4 شاشة نتائج فحص الرابط



الشكل 3-4: نتيجة فحص الرابط والتقرير الأمني المبسط.

تعرض هذه الشاشة النتيجة النهائية لعملية الفحص بعد استلام البيانات من خدمة VirusTotal وتحليلها بواسطة الذكاء الاصطناعي. وتشمل حالة الرابط، والملخص الأمني، والنصائح الوقائية، مع إمكانية الاطلاع على التفاصيل التقنية الخاصة بنتائج VirusTotal أو الانتقال مباشرة إلى الرابط بعد التأكد من سلامته.

## 4-4 سجل عمليات الفحص



الشكل 4-4: واجهة سجل الفحوصات السابقة.

تعرض هذه الشاشة جميع عمليات الفحص التي أجراها المستخدم، مع إمكانية البحث داخل السجل أو تصفية النتائج. كما تُظهر حالة كل عملية فحص، سواء كانت آمنة أو خبيثة، مما يساعد المستخدم على مراجعة النتائج السابقة بسرعة وسهولة.

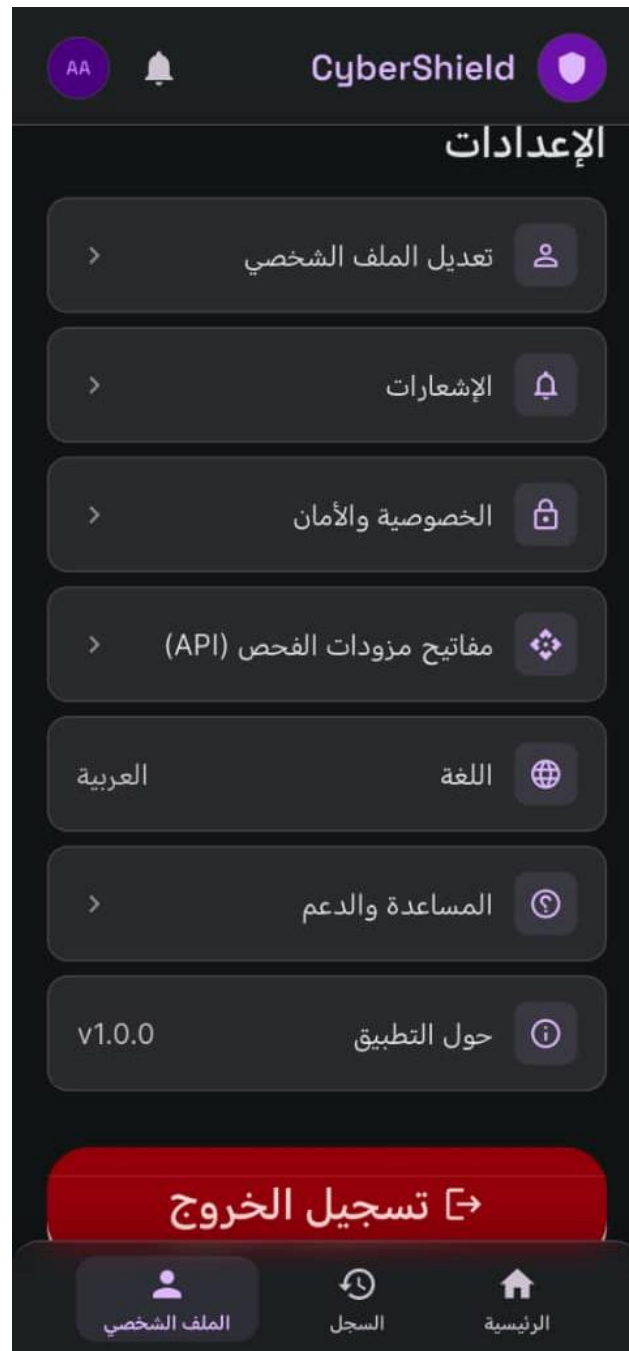
## 5-4 الملف الشخصي للمستخدم



الشكل 5-4: واجهة الملف الشخصي وإحصائيات الاستخدام.

تعرض هذه الشاشة المعلومات الأساسية الخاصة بالمستخدم، إضافة إلى بعض الإحصائيات المتعلقة باستخدام التطبيق، مثل عدد الروابط والملفات التي تم فحصها وعدد التهديدات المكتشفة، مع توفير إمكانية الوصول إلى إعدادات الحساب.

## 6-4 واجهة الإعدادات



الشكل 6-4: واجهة إعدادات التطبيق.

توفر هذه الشاشة مجموعة من الخيارات الخاصة بإدارة التطبيق، مثل تعديل الملف الشخصي، وإدارة الإشعارات، وإعدادات الخصوصية والأمان، وإدارة مفاتيح مزودي خدمات الفحص (API)، واختيار لغة التطبيق، بالإضافة إلى معلومات الإصدار وإمكانية تسجيل الخروج.

#### 7-4 واجهة فحص الروابط قبل الإدخال



الشكل 7-4: واجهة فحص الروابط قبل إدخال عنوان إلكتروني.

تمثل هذه الشاشة الحالة الابتدائية لوظيفة فحص الروابط قبل إدخال أي عنوان إلكتروني، حيث تعرض حقل الإدخال فارغاً مع خيارات الفحص المتاحة وزر بدء الفحص، مما يوضح الشكل الأولي للواجهة قبل بدء عملية التحليل.

# الفصل الخامس: الخاتمة والآفاق المستقبلية

## 5-1 الخاتمة

تناول هذا المشروع تصميم وتنفيذ تطبيق CyberShield، وهو تطبيق مخصص لتعزيز أمن المستخدم عند التعامل مع الملفات والروابط الإلكترونية من خلال دمج تقنيات الأمن السيبراني مع خدمات الذكاء الاصطناعي. وقد اعتمد النظام على بنية حديثة تجمع بين تطبيق الهاتف المحمول والخدمات السحابية، مما أتاح تنفيذ عمليات الفحص بكفاءة مع المحافظة على سهولة الاستخدام.

استُخدم إطار العمل Flutter لتطوير التطبيق، بينما اعتمد الخادم الخلفي على Cloudflare Workers، مع استخدام Cloudflare D1 لإدارة البيانات وCloudflare KV للتخزين المؤقت، كما جرى دمج خدمة VirusTotal API لإجراء عمليات الفحص الأمني، والاستفادة من Google Gemini API لتحويل النتائج التقنية إلى تقارير مبسطة تتضمن ملخصًا أمنيًا وتوصيات تساعد المستخدم على فهم المخاطر واتخاذ القرار المناسب.

وقد أظهر التطبيق قدرة على تنفيذ عمليات فحص الملفات والروابط وعرض نتائجها بصورة واضحة، مع الاحتفاظ بسجل الفحوصات وإدارة حسابات المستخدمين وإعدادات التطبيق ضمن واجهة سهلة الاستخدام. كما ساهم توظيف خدمات الحوسبة السحابية في تبسيط البنية الخلفية وتحسين قابلية التوسع وتقليل متطلبات إدارة الخوادم.

بوجه عام، حقق المشروع الأهداف الموضوعة في بداية الدراسة، وقدم نموذجًا عمليًا يوضح إمكانية توظيف التقنيات السحابية والذكاء الاصطناعي في تطوير تطبيقات أمن سيبراني حديثة توفر نتائج دقيقة وتجربة استخدام مناسبة للمستخدمين.

## 5-2 أهم النتائج

أسفر تنفيذ المشروع عن مجموعة من النتائج، من أهمها:

👤 تطوير تطبيق متكامل لفحص الملفات والروابط الإلكترونية من خلال واجهة استخدام سهلة وبسيطة.

👤 دمج خدمة VirusTotal لإجراء عمليات الفحص والاستفادة من نتائجها الأمنية.

👤 توظيف Google Gemini لتوليد ملخصات أمنية مبسطة وتقديم توصيات وقائية للمستخدم.

👤 بناء خادم خلفي يعتمد على Cloudflare Workers بما يحقق سرعة الاستجابة وسهولة التوسع.

👤 استخدام Cloudflare D1 لإدارة بيانات النظام و Cloudflare KV لتحسين أداء الوصول إلى البيانات.

👤 توفير سجل لعمليات الفحص وإدارة للحساب الشخصي وإعدادات التطبيق.

👤 تطبيق آليات مصادقة آمنة باستخدام (JWT) JSON Web Token لحماية جلسات المستخدم.

### 3-5 الآفاق المستقبلية

يمكن تطوير النظام مستقبلاً بإضافة مجموعة من الخصائص التي تسهم في زيادة كفاءته وتوسيع مجالات استخدامه، ومن أبرزها:

👤 دعم فحص أنواع إضافية من الملفات والوسائط الرقمية.

👤 دمج مزودي فحص أمني إضافيين إلى جانب VirusTotal لزيادة موثوقية نتائج التحليل.

👤 تطوير نموذج ذكاء اصطناعي متخصص في تفسير نتائج الفحص بدلاً من الاعتماد على النماذج العامة.

👤 توفير إشعارات فورية عند اكتشاف تهديدات جديدة أو تحديث نتائج الفحص.

👤 دعم العمل على أنظمة تشغيل إضافية مع الحفاظ على تجربة استخدام موحدة.

إضافة لوحة تحكم إدارية متقدمة لمراقبة استخدام النظام وتحليل الإحصائيات. 🛡️

دعم العمل دون اتصال بالإنترنت لبعض الوظائف الأساسية، مع مزامنة البيانات عند

استعادة الاتصال.

تطوير آليات متقدمة لتحليل السلوك والكشف المبكر عن التهديدات الإلكترونية. 🛡️

دمج النظام مع منصات الحماية المؤسسية وأنظمة إدارة الأمن (SIEM) لدعم بيئات العمل

الكبيرة.

## جدول الاختصارات

| الاختصار | المصطلح الكامل                        | المعنى بالعربية                          |
|----------|---------------------------------------|------------------------------------------|
| AI       | Artificial Intelligence               | الذكاء الاصطناعي                         |
| API      | Application Programming Interface     | واجهة برمجة التطبيقات                    |
| DB       | Database                              | قاعدة البيانات                           |
| D1       | Cloudflare D1 Database                | قاعدة بيانات Cloudflare D1               |
| ERD      | Entity Relationship Diagram           | مخطط الكيانات والعلاقات                  |
| GUI      | Graphical User Interface              | واجهة المستخدم الرسومية                  |
| HTTP     | HyperText Transfer Protocol           | بروتوكول نقل النص التشعبي                |
| HTTPS    | HyperText Transfer Protocol<br>Secure | بروتوكول نقل النص التشعبي الآمن          |
| JSON     | JavaScript Object Notation            | تنسيق تبادل البيانات JSON                |
| JWT      | JSON Web Token                        | رمز ويب بصيغة JSON                       |
| KV       | Key-Value Storage                     | تخزين المفتاح والقيمة                    |
| REST     | Representational State Transfer       | أسلوب نقل الحالة التمثيلية               |
| SHA-256  | Secure Hash Algorithm 256-bit         | خوارزمية التجزئة الآمنة بطول 256 بت      |
| SQL      | Structured Query Language             | لغة الاستعلامات المهيكلية                |
| SQLite   | Structured Query Language Lite        | قاعدة بيانات SQLite                      |
| UI       | User Interface                        | واجهة المستخدم                           |
| URL      | Uniform Resource Locator              | محدد موقع الموارد الموحد                 |
| UUID     | Universally Unique Identifier         | معرف فريد عالميًا                        |
| VT       | VirusTotal                            | خدمة VirusTotal لفحص الملفات<br>والروابط |

## جدول المصطلحات العلمية

| المصطلح العلمي                               | التعريف                                                                                                             |
|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| الأمن السيبراني (Cybersecurity)              | مجموعة التقنيات والإجراءات المستخدمة لحماية الأنظمة والبيانات والمستخدمين من الهجمات والتهديدات الإلكترونية.        |
| الهجمات السيبرانية (Cyber Attacks)           | محاولات تستهدف اختراق الأنظمة أو سرقة البيانات أو تعطيل الخدمات الإلكترونية.                                        |
| برمجيات الفدية (Ransomware)                  | نوع من البرمجيات الخبيثة يقوم بتشفير ملفات الضحية ويطلب فدية مقابل استعادتها.                                       |
| التصيد الاحتيالي (Phishing)                  | هجوم يعتمد على خداع المستخدم للحصول على بياناته الحساسة عبر مواقع أو رسائل مزيفة.                                   |
| هجمات اليوم الصفر (Zero-Day Attacks)         | هجمات تستغل ثغرات أمنية غير معروفة أو لم تُصدّر لها تحديثات أمنية بعد.                                              |
| التعلم العميق (Deep Learning)                | أحد فروع الذكاء الاصطناعي يعتمد على الشبكات العصبية متعددة الطبقات لاستخراج الأنماط المعقدة من البيانات.            |
| الوكيل الذكي (AI Agent)                      | برنامج يعتمد على الذكاء الاصطناعي لتنفيذ المهام وتحليل النتائج وتقديم التوصيات بصورة تلقائية.                       |
| الذكاء الاصطناعي التوليدي (Generative AI)    | تقنيات ذكاء اصطناعي قادرة على إنتاج محتوى جديد مثل النصوص أو الصور أو التوصيات اعتماداً على البيانات المدربة عليها. |
| هندسة التلقين (Prompt Engineering)           | عملية تصميم وصياغة التعليمات الموجهة لنماذج الذكاء الاصطناعي للحصول على نتائج دقيقة وفعالة.                         |
| المعالجة على الجهاز (On-Device Processing)   | تنفيذ عمليات المعالجة مباشرة على جهاز المستخدم دون الاعتماد الكامل على الخوادم السحابية.                            |
| المعالجة السحابية (Cloud Processing)         | تنفيذ عمليات المعالجة باستخدام خوادم سحابية خارجية ذات قدرات حاسوبية عالية.                                         |
| الفحص متعدد المحركات (Multi-Engine Scanning) | أسلوب يعتمد على استخدام عدة محركات كشف أمنية لتحسين دقة اكتشاف التهديدات.                                           |
| التحليل الثابت (Static Analysis)             | تحليل الملفات أو البرامج دون تشغيلها بهدف اكتشاف الثغرات أو البرمجيات الخبيثة.                                      |

## جدول المصطلحات العلمية

|                                                                                                    |                                            |
|----------------------------------------------------------------------------------------------------|--------------------------------------------|
| تحليل البرامج أثناء تشغيلها لمراقبة سلوكها واكتشاف الأنشطة الضارة.                                 | التحليل الديناميكي (Dynamic Analysis)      |
| قيمة رقمية فريدة تمثل الملف وتُستخدم للتحقق من سلامته أو التعرف عليه دون إرسال محتواه.             | البصمة الرقمية (Hash)                      |
| خوارزمية تحول البيانات إلى قيمة ثابتة الطول تُستخدم للتحقق من سلامة البيانات والتعرف عليها.        | خوارزمية التجزئة (Hashing)                 |
| الجزء الذي يتفاعل معه المستخدم لإدخال البيانات وعرض النتائج.                                       | واجهة المستخدم (User Interface - UI)       |
| جودة وسهولة استخدام النظام ومدى رضا المستخدم أثناء التعامل معه.                                    | تجربة المستخدم (User Experience - UX)      |
| بيئة تطوير توفر بنية وأدوات جاهزة لتسهيل بناء التطبيقات البرمجية.                                  | إطار العمل (Framework)                     |
| أسلوب لتنظيم بيانات التطبيق وضمان تحديث واجهات المستخدم عند تغييرها.                               | إدارة الحالة (State Management)            |
| إدارة انتقال المستخدم بين واجهات التطبيق المختلفة بطريقة منظمة.                                    | التنقل بين الشاشات (Navigation Management) |
| آلية تسمح بالانتقال المباشر إلى صفحة محددة داخل التطبيق من خلال رابط خارجي.                        | الروابط العميقة (Deep Linking)             |
| تحويل المستخدم تلقائياً إلى شاشة أو مسار معين وفق شروط محددة.                                      | إعادة التوجيه (Redirection)                |
| آلية لمعالجة الطلبات والاستجابات قبل إرسالها أو بعد استقبالها لأغراض الأمان أو التعديل.            | الاعتراضات (Interceptors)                  |
| حفظ البيانات الحساسة باستخدام وسائل تشفير تمنع الوصول غير المصرح به.                               | التخزين الآمن (Secure Storage)             |
| تخزين البيانات مؤقتاً لتسريع الوصول إليها وتقليل عدد الطلبات إلى الخادم.                           | التخزين المؤقت (Caching)                   |
| الجزء المسؤول عن معالجة البيانات وتنفيذ منطق الأعمال والتواصل مع قواعد البيانات والخدمات الخارجية. | الخادم الخلفي (Backend)                    |
| نظام منظم لتخزين البيانات وإدارتها واسترجاعها.                                                     | قاعدة البيانات (Database)                  |

## جدول المصطلحات العلمية

|                                        |                                                                                     |
|----------------------------------------|-------------------------------------------------------------------------------------|
| الخدمات السحابية (Cloud Services)      | خدمات حاسوبية تُقدم عبر الإنترنت مثل التخزين والمعالجة وقواعد البيانات.             |
| المصادقة (Authentication)              | عملية التحقق من هوية المستخدم قبل السماح له باستخدام النظام.                        |
| التحقق من الصلاحيات (Authorization)    | تحديد العمليات والموارد المسموح للمستخدم بالوصول إليها بعد تسجيل الدخول.            |
| زمن الاستجابة (Latency)                | الزمن المستغرق بين إرسال الطلب واستقبال الاستجابة.                                  |
| استهلاك الموارد (Resource Consumption) | مقدار استخدام التطبيق لموارد الجهاز مثل المعالج والذاكرة والطاقة.                   |
| البرمجيات الخبيثة (Malware)            | برامج ضارة تهدف إلى إلحاق الضرر بالأنظمة أو سرقة البيانات أو التجسس على المستخدمين. |

## REFERENCES:

- [1] Zeng, W., et al. (2024). Mobile device usage statistics and security implications. *IEEE Access*, 12, 93575–93600.
- [2] Google Security Team. (2024). Android security ecosystem report 2024. Google Security Blog.
- [3] Gupta, B. B., et al. (2024). Cyberattacks and security of cloud computing: A complete guideline. *Symmetry*, 15(11), 1981.
- [4] Chandran, A. L., et al. (2025). A comparative analysis of machine learning models on detecting malware in Android devices. *Journal of Cyber Security and Risk Auditing*, 4, 327–346.
- [5] Hurry-sh, & CCNCS Project. (2025). Detection of mobile malware (Android) using ML and hybrid analysis. In *Proceedings of the 7th Asia Pacific Information Technology Conference* (pp. 1–8). ACM.
- [6] Onwuzurike, L., et al. (2019). Mamadroid: Detecting Android malware by building Markov chains of behavioral models (extended version). *ACM Transactions on Privacy and Security*, 22(2), 1–34.
- [7] Kumar, A., & Sharma, V. (2025). A hybrid super learner ensemble for phishing detection on mobile devices. *PMC*.

- [8] Hassan, M., et al. (2025). A robust and dynamic malware detection and classification model using behavioral-based analysis and BERT technique. PMC.
- [9] Heiding, F., et al. (2024). Devising and detecting phishing emails using large language models. IEEE Access, 12, 42131–42146.
- [10] Jain, A. K., & Gupta, B. B. (2018). Rule-based framework for detection of smishing messages in mobile environment. Procedia Computer Science, 125, 617–623.
- [11] Jošt, G., & Taneski, V. (2025). State-of-the-Art Cross-Platform Mobile Application Development Frameworks: A Comparative Study of Market and Developer Trends. Informatics, 12(2), 45. <https://doi.org/10.3390/informatics12020045>
- [12] Kinari, S. A., et al. (2024). An Independent Learning System for Flutter Cross-Platform Application Development. Information, 15(10), 614. <https://doi.org/10.3390/information15100614>
- [13] Puryanto, J. A., & Akbar, H. (2025). Performance Analysis of Provider and Riverpod State Management Library on Flutter Applications. Journal of Technology and Informatics, 7(2), 190–200.
- [14] Rousselet, R. (2025). Riverpod Documentation and Concepts. Riverpod Official Documentation. <https://riverpod.dev> Retrieved April 18, 2026, from <https://riverpod.dev>

- [15] Banik, S., & Roy, S. (2023). Navigation Management in Cross-Platform Flutter Applications: A Comparative Study. International Journal of Advanced Computer Science and Applications (IJACSA).
- [16] Flutter Team. (2024). GoRouter: Declarative Routing for Flutter. Flutter Package Documentation. Retrieved May 6, 2026, from [https://pub.dev/packages/go\\_router](https://pub.dev/packages/go_router)
- [17] Kumar, A., & Sharma, R. (2023). Performance Evaluation of HTTP Client Libraries for Flutter Applications. International Journal of Advanced Computer Science and Applications (IJACSA).
- [18] Alshammari, M., Alenezi, M., & Alrashidi, A. (2024). Best Practices for RESTful API Integration in Cross-Platform Mobile Applications. Journal of King Saud University – Computer and Information Sciences.
- [19] Pacheco, J., & Silva, R. (2023). Secure Storage Mechanisms for Mobile Applications: A Comparative Study. IEEE Access.
- [20] OWASP Foundation. (2024). OWASP Mobile Application Security Testing Guide (MASTG): Secure Local Storage. Retrieved May 19, 2026, from <https://mas.owasp.org/>
- [21] Cloudflare. (2024). Cloudflare Workers Documentation. Cloudflare Developers. Retrieved April 24, 2026, from <https://developers.cloudflare.com/workers/>

- [22] Satoh, I. (2023). Serverless Computing for Edge Applications: Architectures and Design Considerations. IEEE Access, 11, 118420–118437.
- [23] Kavis, M. J. (2023). Cloud-Native Data Management for Modern Applications. IEEE Cloud Computing.
- [24] Cloudflare. (2024). Cloudflare D1 Documentation. Cloudflare Developers. Retrieved June 2, 2026, from <https://developers.cloudflare.com/d1/>
- [25] Cloudflare. (2024). Cloudflare Workers KV Documentation. Cloudflare Developers. Retrieved June 15, 2026, from <https://developers.cloudflare.com/kv/>
- [26] Satyanarayanan, M., et al. (2023). Edge Computing and Distributed Storage Systems: Principles and Applications. IEEE Internet Computing.
- [27] VirusTotal. (2024). VirusTotal API Documentation. VirusTotal Official Documentation. Retrieved July 3, 2026, from <https://docs.virustotal.com/>
- [28] Zhu, Y., et al. (2024). Collaborative Malware Detection Using Multi-Engine Threat Intelligence Platforms. Future Internet, 16(8), 259.
- [29] Google DeepMind. (2024). Gemini API Documentation. Google AI Developers. Retrieved May 28, 2026, from <https://ai.google.dev/>
- [30] Team Gemini. (2024). Gemini: A Family of Highly Capable Multimodal Models. arXiv preprint. <https://arxiv.org/abs/2312.11805>

[31] Jones, M., Bradley, J., & Sakimura, N. (2021). JSON Web Token (JWT): Best Current Practices. Internet Engineering Task Force (IETF). RFC 8725.

[32] Alshammari, M., Alenezi, M., & Alrashidi, A. (2024). Secure Authentication Mechanisms for RESTful Web Services. Journal of King Saud University – Computer and Information Sciences.

Syrian Arab Republic

Ministry of Higher Education and Scientific Research

Al-Wataniya Private University

Faculty of Engineering

Department of Computer and Communications Engineering



Graduation Project Submitted in Partial Fulfillment of the Requirements for the  
Bachelor's Degree in Computer and Communications Engineering

## CyberShield

An Artificial Intelligence-Powered Application for File and URL  
Scanning

Prepared by:

**Alaa Halabi**

**Amani Al-Amin**

**Heba Al-Fares**

Supervised by:

**Dr. Ahmad Al-Abdo**

**Dr. Mahmoud Ayyoub**

Academic Year 2025–2026